

Computing Secure and Efficient Parameters for Lattice-Based Cryptographic Schemes

Bachelor's Thesis

in Partial Fulfillment of the Requirements for the
Degree of
Bachelor of Science

by
KONSTANTIN GUBENKO

submitted to:
Prof. Dr. Johannes Blömer
and
Prof. Dr.-Ing. Juraj Somorovsky

Paderborn, September 1, 2025

Contents

1	Introduction	1
2	Mathematical Foundations	3
2.1	Notation	3
2.2	Lattices	3
2.3	Learning With Errors	6
2.4	Ring-LWE	7
2.5	Module-LWE	8
2.6	Short Integer Solution	9
2.7	Ring-SIS	10
2.8	Module-SIS	11
3	Attacks and Hardness Estimation for LWE and SIS	13
3.1	Lattice Estimator by Albrecht	13
3.2	Primal Attacks on LWE	15
3.3	Other Attacks on LWE	19
3.4	Attacks on SIS	25
4	Development of the Parameter Selection Program	29
4.1	Main Goal and Motivation	29
4.2	Basic Functionality and Design Decisions	29
4.3	Advanced Functionality	34
5	Evaluation	43
5.1	Criteria and Improvements	43
5.2	Practical Implications.	44
6	Installation requirements	45
	Bibliography	47

1 Introduction

Motivation. It is widely believed that lattice-based cryptographic constructions based on problems such as Learning With Errors (LWE) and Short Integer Solution (SIS) offer strong hardness guarantees and are expected to resist known quantum algorithms. In practice, however, deploying a lattice-based scheme requires us to select concrete parameter values that are both secure and efficient, which often is not trivial to perform. In essence, good parameters keep the scheme fast while making relevant attacks computationally infeasible. Our thesis addresses that step.

Foundational hardness results and worst-case to average-case reductions (e.g., for LWE) show that, for sufficiently large parameters, solving certain lattice problems is computationally hard. However, they do not tell us which exact values to choose for a given security level and implementation constraints. Moreover, different attack families react differently to parameter changes, so parameter choice involves several trade-offs at once. While proven, asymptotic and conservative bounds provide important guidance, they are often too loose for tuning concrete instances. In practice, we need tighter, empirically based estimates for specific instances. The cryptographic community therefore uses the Lattice Estimator by Albrecht and his colleagues [APS15] to complement theoretical proofs and approximate the security level of a concrete instance against a list of well-studied attacks.

In deployment, we need the inverse of the estimator’s usual workflow: given a target bit security and user-specified constraints, find concrete parameters that meet the target bit security against the relevant attacks, while keeping computation overhead small. A naive brute-force search is not the right method here. Each estimator evaluation is relatively expensive, especially for larger numbers. Huge domains make exhaustive brute-force enumeration infeasible. Furthermore, parameters interact in complex ways across different attacks and relate to each other in a tangled way. Manual probing across multiple attacks and constraints is also slow and easy to do incorrectly.

The main goal of this thesis is to design an automatic method that turns a security target and constraints into concrete parameter values for LWE and SIS and their variants. The implemented program should be also user-friendly, with clear inputs and outputs and reproducible runs.

Main idea. Let us provide the high level intuition behind our approach. Our tool is essentially a Python command-line program that takes all required inputs, including structural constraints. Instead of brute force, we search the parameter space intelligently. Namely, we model constraints symbolically and use the Z3 SMT solver by Microsoft [DMB08] to generate promising candidate parameter sets. We then evaluate

their security with the lattice estimator and iterate in a loop, tightening or expanding the search space only where needed (for specific parameters). The program is flexible: it allows tuning problem-specific parameters, several program constants, and supports additional features such as optimization goals and strategies.

Having introduced our goal and motivation, we now present the mathematical foundations required for the rest of this thesis.

2 Mathematical Foundations

2.1 Notation

We use bold lowercase letters to denote column vectors (e.g., $\mathbf{a}$) and angle brackets to denote inner product of two vectors (e.g., $\langle \mathbf{a}, \mathbf{b} \rangle$). The transpose of a vector $\mathbf{a}$ is written as $\mathbf{a}^t$, which is a row vector.

With $\|\cdot\|$ we denote the Euclidean norm, unless otherwise specified. Bold uppercase letters (e.g., $\mathbf{B}$) denote matrices.

We denote by $\mathbb{Z}_q$ the quotient ring of integers modulo a positive integer q . It consists of the set $\{0, 1, \dots, q-1\}$, where operations (addition and multiplication) are performed modulo q . This defines a finite ring widely used in cryptography for its efficiency due to the efficiency of modular arithmetic.

2.2 Lattices

Before we dive into lattice-based cryptography and its underlying problems, let us first look what a lattice is, provide some motivation and intuition, and give formal definitions of it.

For better understanding at this point, think of a lattice as a discrete grid of points in some Euclidean space, generated by linear combinations of a set of linearly independent vectors with exclusively integer coefficients with useful algebraic structure.

At the very beginning of the 19th century the lattice concept had not yet been formalised, lattices were already studied in number theory to understand integer solutions to special equations, called Diophantine equations, by Lagrange and Gauss in [Gau01]. Later in the century many other scientists have contributed to the modern foundation for lattice theory [Ded79, Min96]. Lattices have found their application also in group theory [CS13], crystallography [Bra66], physics [Wal56].

Now we can define lattices formally and provide further definitions tied to it.

Definition 2.1 (Lattice (1)) *For $n \geq 1, n \in \mathbb{Z}$, an n -dimensional lattice $\mathcal{L}$ is a discrete additive subgroup of $\mathbb{R}^n$, where $\mathbb{R}$ denotes an Euclidean space.*

To recall, an additive subgroup is a subset of group that (1) is closed under addition, (2) has the identity element, and (3) each element of the subgroup has one additive inverse. Discreteness of the lattice $\mathcal{L}$ ensures that there is always some space between points. So $\forall x \in \mathcal{L}$ there is an open ball with some radius $r \neq 0$, in which x would be the only one point in $\mathcal{L}$.

As mentioned above, we can define a lattice in terms of linear combinations of a set of linearly independent vectors, known as a basis.

Definition 2.2 (Lattice (2)) *For $k, n \geq 1$, $k, n \in \mathbb{Z}$, $k \leq n$, let $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k\}$ be linearly independent in $\mathbb{R}^n$. We define the k -rank, n -dimensional lattice as*

$$\mathcal{L}(\mathbf{B}) = \mathcal{L}(\mathbf{b}_1, \dots, \mathbf{b}_k) = \left\{ \sum_{i=1}^k z_i \mathbf{b}_i \mid z_i \in \mathbb{Z} \right\}.$$

For the rest of the thesis, we focus exclusively on full-rank lattices (i.e. $k = n$).

A basis $\mathbf{B}$ is not unique for a given lattice $\mathcal{L}$, meaning that the same lattice can have multiple different bases. We can say that two different bases $\mathbf{B}_1$ and $\mathbf{B}_2$ define the same lattice $\mathcal{L}$ if and only if there exists a unimodular matrix $\mathbf{U} \in \mathbb{Z}^{n \times n}$ (a square integer matrix with $\det(\mathbf{U}) = \pm 1$) such that $\mathbf{B}_1 = \mathbf{U} \cdot \mathbf{B}_2$. Since a lattice can have many different bases, in practice we can distinguish between "good" and "bad" bases. To obtain a more useful basis, we can apply a basis reduction algorithm. Basis reduction is the process of obtaining a reduced basis for a lattice, using specific reduction algorithms like LLL or BKZ [NV10, CN11]. The shorter and more orthogonal the vectors, the better the basis for solving the problem.

In Figure 2.1, we see a lattice $\mathcal{L}_1$ generated by a basis $\mathbf{B}_1$. In Figure 2.2, the same lattice $\mathcal{L}_1$ is shown, but now with a different basis $\mathbf{B}_2$. Figure 2.3 illustrates a new lattice $\mathcal{L}_3$, generated by a basis $\mathbf{B}_3$.

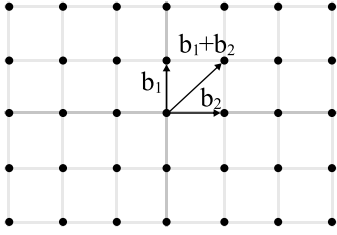


Figure 2.1

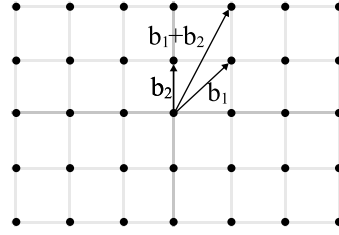


Figure 2.2

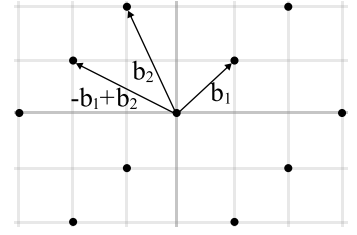


Figure 2.3

The length of the shortest nonzero vector in a lattice $\mathcal{L}$ defines another important lattice property called the minimum distance. The formal definition is provided below.

Definition 2.3 (The minimum distance) *We define the minimum distance $\lambda_1(\mathcal{L})$ of a lattice $\mathcal{L}$ as $\lambda_1(\mathcal{L}) = \min_{\mathbf{x} \in \mathcal{L} \setminus \{\mathbf{0}\}} \|\mathbf{x}\|$.*

Having defined lattices in general, we can now introduce certain special and particularly relevant for us lattices, that will be needed further in the discussion of known lattice-based attacks.

The dual lattice. We define the dual lattice $\mathcal{L}^*$ of a lattice $\mathcal{L}$ as the set of all vectors $\mathbf{y} \in \mathbb{R}^n$ whose inner product with every vector from $\mathcal{L}$ is an integer. We also provide a formal definition.

Definition 2.4 (The dual lattice [Reg04]) *We define the dual $\mathcal{L}^*$ of a lattice $\mathcal{L} \subset \mathbb{R}^n$ as*

$$\mathcal{L}^* = \{\mathbf{y} \in \mathbb{R}^n \mid \forall \mathbf{x} \in \mathcal{L}, \langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z}\}.$$

For example, for the lattice $\mathcal{L}_1 = \mathbb{Z}^2$ with basis $\mathbf{b}_1 = (1, 0)$, $\mathbf{b}_2 = (0, 1)$, its dual is $\mathcal{L}_1^* = \mathbb{Z}^2$. Every vector with integer coordinates has an integer inner product with all points of $\mathcal{L}_1(\mathbf{b}_1, \mathbf{b}_2)$, and no other vectors do.

For the lattice $\mathcal{L}_2$ with basis $\mathbf{b}'_1 = (\frac{1}{2}, 0)$, $\mathbf{b}'_2 = (0, \frac{1}{2})$, its dual lattice is $\mathcal{L}_2^* = \{(2k, 2l) \mid k, l \in \mathbb{Z}\}$

The ideal lattice. Another special class of lattices important for our survey is the class of ideal lattices. These lattices come from ideals in polynomial rings studied in algebraic number theory. Simply put, an ideal lattice $\mathcal{L}$ is constructed from special vectors called coefficient vectors, which come from all polynomials belonging to a specific ideal within such a ring. Let us define it formally.

Remark 2.5 *A coefficient vector $\mathbf{v}$ obtained from a polynomial $p(x)$ is defined as $\mathbf{v} = (p_0, p_1, \dots, p_{n-1})$, where $p(x) = \sum_{i=0}^{n-1} p_i x^i$.*

Definition 2.6 (The ideal lattice [LPR10]) *Let $R = \mathbb{Z}[x]/f(x)$ be a polynomial ring for some irreducible polynomial $f(x)$, and let $I \subseteq R$ be an ideal. The ideal lattice $\mathcal{L}(I)$ is defined as*

$$\mathcal{L}(I) = \{v(p) \mid p(x) \in I\},$$

where $v(p)$ denotes the coefficient vector of the polynomial $p(x)$.

Remark 2.7 *The Euclidean norm of a ring vector $v \in R^m$, where each component is a polynomial represented by its coefficient vector, is defined as the Euclidean norm of the concatenation of all coefficient vectors. Formally, $\|\mathbf{v}\| = \sqrt{\sum_{i=1}^m \sum_{j=0}^{n-1} v_{i,j}^2}$, where $v_{i,j}$ is the j -th coefficient of the i -th polynomial component of $\mathbf{v}$.*

Ideal lattices play the crucial role in lattice-based cryptography, because their structure allows for more efficient computations and smaller key sizes compared to general lattices [LPR10].

The module lattice. Just as ideal lattices are built from ideals in polynomial rings, we can also define more general structure called a module lattice. A module is an algebraic structure that generalizes the concept of a vector space by replacing the field of scalars with a ring. Modules can be seen as vectors with coefficients in a ring. Ideals are special cases of modules.

Module lattices are built from modules over polynomial rings instead of using just ideals. In essence, a module lattice $\mathcal{L}$ is constructed from the coefficient vectors of all elements belonging to a particular module in a ring. Now we can define it formally.

Definition 2.8 (The module lattice [LS14]) *Let $R = \mathbb{Z}[x]/f(x)$ be a polynomial ring for some irreducible polynomial $f(x)$, and let $M \subseteq R^d$ be a module. The module lattice $\mathcal{L}(M)$ is defined as*

$$\mathcal{L}(M) = \{v(\mathbf{p}) \mid \mathbf{p} \in M\},$$

where $v(\mathbf{p})$ denotes the concatenation of the coefficient vectors of all d polynomials in $\mathbf{p}$.

Remark 2.9 *For a polynomial ring R of a degree n and a module $M \subseteq R^d$, each element $\mathbf{p} \in M$ is a vector of d polynomials and is defined as $\mathbf{p} = (p_1(x), p_2(x), \dots, p_d(x))$, where $p_i(x) = \sum_{j=0}^{n-1} a_{i,j}x^j$ for $i = 1, \dots, d$, $a_{i,j} \in \mathbb{Z}$.*

Module lattices offer greater flexibility in cryptographic constructions [LS14].

Having introduced special lattices, we now present important computational problems defined over lattices. Many such problems, including the Shortest Vector Problem (SVP), the Closest Vector Problem (CVP), and the Bounded Distance Decoding Problem (BDD), are known to be computationally hard (NP-hard), even against quantum attacks [Pei16]. For the scope of this thesis, we focus exclusively on the Shortest Vector Problem.

Definition 2.10 (Shortest Vector Problem [Pei16]) *Given an arbitrary basis $\mathbf{B}$ of some lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$, find a shortest nonzero lattice vector, i. e., a $\mathbf{v} \in \mathcal{L}$ for which $\|\mathbf{v}\| = \lambda_1(\mathcal{L})$.*

Intuitively, SVP asks us to find the shortest vector from the origin to any other lattice point. The problem quickly becomes extremely challenging in higher dimensions, and no known algorithms efficiently solve SVP when the lattice dimension is sufficiently large.

This computational hardness has important implications. Ajtai [Ajt96] has shown the connection between worst-case for SVP and average-case hardness for lattices. This result forms the basis for certain mathematical problems in cryptography, which we define further.

2.3 Learning With Errors

Building on Ajtai's results, in [Reg09] Regev has introduced the Learning With Errors (LWE) problem, that serves as a foundational computational problem in lattice-based cryptography. Regev proved that solving LWE in the average case in some dimension n is as hard as solving worst-case lattice problems such as SVP in any n -dimensional lattice. In other words, this reduction guarantees that breaking LWE-based cryptography requires solving all instances of the hard lattice problems SVP. Also Regev showed that LWE resists known quantum attacks.

Before defining the LWE problem, we define the LWE Distribution, which is parameterized by three parameters: the dimension n and modulus q , both positive integers, and a discrete error distribution χ .

Definition 2.11 (LWE Distribution [NHAJRM24]) *For a vector $\mathbf{s} \in \mathbb{Z}_q^n$ called the secret, the LWE distribution $A_{\mathbf{s},\chi}$ over $\mathbb{Z}_q^n \times \mathbb{Z}_q$ is sampled by choosing $\mathbf{a} \in \mathbb{Z}_q^n$ uniformly at random, choosing $e \leftarrow \chi$, and outputting $(\mathbf{a}, b = \langle \mathbf{s}, \mathbf{a} \rangle + e \pmod{q})$, where $\langle \mathbf{s}, \mathbf{a} \rangle$ denotes the inner product of the vectors $\mathbf{s}$ and $\mathbf{a}$.*

In essence, for the chosen fixed secret vector $\mathbf{s}$ and some sampled uniformly at random vector $\mathbf{a}$ from the same quotient ring, we compute its inner product, slightly distort it by adding a small random noise e from the error distribution, and reduce the result modulo q . This gives us a noisy linear equation of the form $b = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}$. Each such pair $(\mathbf{a}, b)$ is called an LWE sample. It looks like a linear equation - but one that is slightly corrupted and, therefore, difficult to solve.

Now that we have introduced the LWE distribution, we can define the computational problems based on it. There are two main types of the LWE problem - *Search* and *Decision*. They are parametrized by the number m of samples - noisy linear equations from the LWE distribution. The first one is to find the secret vector given LWE samples and the second one is to decide, whether the given samples are LWE samples or uniformly random.

Definition 2.12 (Search-LWE $_{q,n,m,\chi}$ [Pei16]) *Given m independent samples $(\mathbf{a}_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ drawn from $A_{\mathbf{s},\chi}$ for a uniformly random $\mathbf{s} \in \mathbb{Z}_q^n$ (fixed for all samples), find $\mathbf{s}$.*

Definition 2.13 (Decision-LWE $_{q,n,m,\chi}$ [Pei16]) *Given m independent samples $(\mathbf{a}_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ where every sample is distributed according to either: (1) $A_{\mathbf{s},\chi}$ for a uniformly random $\mathbf{s} \in \mathbb{Z}_q^n$ (fixed for all samples), or (2) the uniform distribution, distinguish which is the case (with non-negligible advantage).*

Even non-quantum computers nowadays are able to solve large systems of linear equations efficiently. However, even a small random error terms added to the equations corrupts the exact linear equations and makes it infeasible for the known non-quantum/quantum algorithms.

2.4 Ring-LWE

Although the standard LWE problem is proven to resist known quantum attacks [Reg09], it operates over large and, what is more important, unstructured vectors. The lack of any underlying algebraic structure leads to large key and ciphertext sizes and slower cryptographic schemes for practical applications. To address these limitations in [LPR10] the Ring-Based LWE was introduced. RLWE resolves the limitations by employing polynomial-ring arithmetic, significantly improving computational efficiency.

The core idea is to replace vectors with polynomials and inner products with polynomial multiplication. Typically RLWE operates over polynomials modulo a cyclotomic polynomial (special polynomial with roots that are powers of a primitive root of unity), used to define a cyclic algebraic structure. Commonly, RLWE is instantiated with ideals in a ring, which enables more efficient arithmetic. The security of RLWE is also reducible to worst-case problems over ideal lattices [LPR10].

To define the RLWE distribution, we need a ring R of degree n over the integers (typically we take $R = \mathbb{Z}[x]/\Phi_n(x)$, where $\Phi_n(x)$ is the n -th cyclotomic polynomial), a positive integer modulus q , and an error distribution χ over R .

Definition 2.14 (Ring-LWE Distribution [Pei16]) *For an $s \in R_q$ called the secret, the ring-LWE distribution $A_{s,\chi}$ over $R_q \times R_q$ is sampled by choosing $a \in R_q$ uniformly at random, choosing $e \leftarrow \chi$, and outputting $(a, b = s \cdot a + e \bmod q)$.*

In essence, the idea mirrors the LWE construction: we choose a secret s (now a polynomial), multiply it by a public random polynomial a , add a random small noise polynomial e , and perform all arithmetic modulo q . The result is another polynomial b , and the pair (a, b) is called a sample. Each sample encodes a noisy polynomial equation, just like LWE samples encoded noisy linear equations.

Like LWE, RLWE has two main versions of the problem, both parameterized by m samples:

Definition 2.15 (Decision-RLWE $_{q,R,m,\chi}$ [Pei16]) *Given m independent samples $(a_i, b_i) \in R_q \times R_q$ where every sample is distributed according to either: (1) $A_{s,\chi}$ for a uniformly random $s \in R_q$ (fixed for all samples), or (2) the uniform distribution, distinguish which is the case (with non-negligible advantage).*

Definition 2.16 (Search-RLWE $_{q,R,m,\chi}$) *Given m independent samples $(a_i, b_i) \in R_q \times R_q$ drawn from $A_{s,\chi}$ for a uniformly random $s \in R_q$ (fixed for all samples), find s .*

In comparison to LWE, which requires $\mathcal{O}(n^2)$ operations per multiplication, RLWE reduces this to $\mathcal{O}(n \log n)$ by leveraging Fast Fourier Transform-like methods [Für07]. While the algebraic structure of RLWE can make it slightly more vulnerable than standard structureless LWE, carefully chosen parameters ensure that RLWE remains hard enough to resist all known quantum attacks. Specifically, there are no known methods for exploiting the algebraic structure in a way that leads to an efficient solution of the problem [WX22, ABD⁺19].

2.5 Module-LWE

While RLWE resolves the main shortcomings of standard LWE by leveraging algebraic structure, it suffers from a lack of flexibility and introduces potential security weaknesses. Module-LWE addresses these limitations and was first introduced in [BGV14], as a generalization of both LWE and RLWE. While LWE operates over $\mathbb{Z}_q$ vectors and RLWE over polynomials, MLWE operates over vectors of polynomials - modules (a mix of structured rings and unstructured vectors), leveraging the advantages of both LWE and RLWE. The security of MLWE can be reduced to worst-case problems on module lattices [LS14].

As before we can now define MLWE formally. To define the MLWE distribution, we require the same parameters as for RLWE, along with one additional positive integer parameter d , called the module rank (the dimension of the module), introducing more flexible parameter choice.

Definition 2.17 (Module-LWE Distribution [AD17]) *For an $\mathbf{s} \in R_q^d$ called the secret, the module-LWE distribution $A_{\mathbf{s},\chi}$ over $R_q^d \times R_q$ is sampled by choosing $\mathbf{a} \in R_q^d$ uniformly at random, choosing $e \leftarrow \chi$, and outputting $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q})$.*

In essence, the core idea is similar to LWE and RLWE. For the chosen secret vector $\mathbf{s}$, consisting of d polynomials from a ring R_q , and some sampled uniformly at random vector $\mathbf{a}$ of the same form and from the same ring, we compute its inner product and add a small random error polynomial e from the error distribution. After reducing the result modulo q , we obtain another polynomial b . The pair $(\mathbf{a}, b)$ is called an MLWE sample. Intuitively, this can be seen as taking multiple RLWE samples and packing them into a single structured framework.

Having defined the MLWE distribution, exactly as for its predecessors, we can now define the corresponding computational problems.

Definition 2.18 (Search-MLWE $_{q,R,d,m,\chi}$) *Given m independent samples $(\mathbf{a}_i, b_i) \in R_q^d \times R_q$ drawn from $A_{\mathbf{s},\chi}$ for a uniformly random $\mathbf{s} \in R_q^d$ (fixed for all samples), find $\mathbf{s}$.*

Definition 2.19 (Decision-MLWE $_{q,R,d,m,\chi}$) *Given m independent samples $(\mathbf{a}_i, b_i) \in R_q^d \times R_q$ where every sample is distributed according to either: (1) $A_{\mathbf{s},\chi}$ for a uniformly random $\mathbf{s} \in R_q^d$ (fixed for all samples), or (2) the uniform distribution, distinguish which is the case (with non-negligible advantage).*

LWE and RLWE can be seen as special cases of MLWE. In particular, MLWE with $d = 1$ is essentially Ring-LWE, while MLWE over $\mathbb{Z}_q$ with module rank d is essentially standard LWE. Module-LWE enhances flexibility by introducing more flexible parameter choice, enabling trade-offs between security and efficiency and providing a fine-tuned balance for different schemes based on it. In a pure RLWE instantiation to benefit from fast multiplication, we typically use a ring degree n that is a power of two. Consequently, if we need to raise the security level even slightly, we are forced to jump to the next power of two for n , roughly doubling the cost of every polynomial multiplication. By contrast, in MLWE we can keep the ring degree fixed and simply increment the module rank d to obtain the required security level with only modest increase in computation [BDK⁺18].

For example, it serves as the underlying problem for fully homomorphic encryption, that allows computations to be performed directly on encrypted data, without needing to decrypt it [BGV14].

2.6 Short Integer Solution

Having introduced the Learning With Errors problem, we now turn to another foundational computational problem closely related to lattice-based cryptography - the Short Integer Solution (SIS) problem. SIS was introduced by Ajtai [Ajt96] for the purpose to establish a connection between the average-case hardness and worst-case hardness of lattice problems. In his work he proved that solving SIS for random instances is as hard

as solving worst-case instances of SVP.

Unlike LWE, SIS is already presented as a search problem. So there are no distinct search and decision variants of the problem. To define SIS, we require several parameters similar to LWE: the dimension n , modulus q , the additive group $\mathbb{Z}_q^n$. Additionally, we need a norm bound β and a number m of samples. This norm bound plays a crucial role in determining the allowed size of solutions. Now we can introduce the formal definition of SIS.

Definition 2.20 (SIS $_{q,n,m,\beta}$ [Pei16]) *Given m uniformly random vectors $\mathbf{a}_i \in \mathbb{Z}_q^n$, forming the columns of a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, find a nonzero integer vector $\mathbf{z} \in \mathbb{Z}^m$ of norm $\|\mathbf{z}\| \leq \beta$ such that*

$$f_{\mathbf{A}}(\mathbf{z}) := \mathbf{A}\mathbf{z} = \sum_i \mathbf{a}_i \cdot z_i = \mathbf{0} \in \mathbb{Z}_q^n.$$

In essence, SIS asks us to find a vector $\mathbf{z}$ that is small enough according to the chosen norm bound β , such that when multiplied by a given chosen uniformly at random matrix $\mathbf{A}$, it results in the zero vector modulo q .

Without the requirement that the norm bound of the solution vector $\mathbf{z}$ be small, the problem becomes trivial, as large solutions are easy to find computationally.

Another important aspect is that, to ensure that the problem is both solvable (i.e., has a non-trivial solution) and is interesting for us, the number m of given samples must be sufficiently large and at the same time the chosen norm bound β cannot be too small.

2.7 Ring-SIS

While standard SIS is a powerful foundational problem in lattice-based cryptography, its practical use is limited by large key sizes and slower computations, just like the challenges that standard LWE faces. To overcome these limitations Ring-Based SIS was introduced in [PR06]. The main goal of RSIS is to leverage algebraic structure to make computations faster and more practical compared to the standard structureless version of SIS problem, the same way RLWE improves LWE.

In simple terms, RSIS operates over polynomials in a ring, instead of plain vectors. It is based on ideal lattices that have repeating (cyclic) structure. Typically, RSIS benefits from FFT-like techniques for polynomial multiplication, significantly speeding up computations and improving efficiency [KNK20], just as RLWE does.

To formally define RSIS, we use similar parameters to those of RLWE: a ring R of degree n (typically we take $R = \mathbb{Z}[x]/\Phi_n(x)$, where $\Phi_n(x)$ is the n -th cyclotomic polynomial), modulus q , a norm bound β , and the number of samples m . The RSIS problem can be stated as follows:

Definition 2.21 (RSIS $_{q,R,m,\beta}$ [Pei16]) *Given m uniformly random elements $a_i \in R_q$, defining a vector $\mathbf{a} \in R_q^m$, find a nonzero vector $\mathbf{z} \in R^m$ of norm $\|\mathbf{z}\| \leq \beta$ such that*

$$f_{\mathbf{a}}(\mathbf{z}) := \langle \mathbf{a}, \mathbf{z} \rangle = \mathbf{a}^t \cdot \mathbf{z} = \sum_i a_i \cdot z_i = 0 \in R_q.$$

Simply put, the RSIS problem asks us to find a short vector $\mathbf{z}$ with a norm smaller than a given bound β so that when we take the inner product with the given vector $\mathbf{a}$, the result is zero in the ring R_q .

The hardness of RSIS is reducible to the SVP on ideal lattices [Pei15, LPR10]. Thanks to the algebraic structure, in comparison with the standard SIS, key sizes of cryptographic schemes based on Ring-SIS are smaller and cryptographic schemes based on RSIS are significantly more efficient in practice.

2.8 Module-SIS

Although RSIS provides efficiency gains over standard SIS, its practical application is restricted due to limited flexibility and possible security risks because of its specific algebraic structure. To address these restrictions and to bridge classic SIS with its ring version RSIS, Module-SIS was introduced in [LS14].

The MSIS problem is considered to be computationally at least as hard as the RSIS problem, providing stronger security guarantees. Specifically, the problem to which MSIS is reduced (Mod-SIVP) is possibly harder than the problem to which RSIS is reduced (Id-SIVP) [LS14]. Moreover, no converse reduction from the Id-SIVP to Mod-SIVP is currently known.

As we have already mentioned, a module is an algebraic structure that generalizes both rings (RSIS) and vector (SIS) spaces, making its concept broader. Exactly as MLWE, MSIS operates over modules - vectors of polynomials, which bring the same benefits for the SIS problem as modules bring to LWE in the terms of flexibility of vectors with the computational efficiency of polynomial rings.

MSIS uses parameters similar to MLWE: a ring R of degree n (usually $R = \mathbb{Z}[x]/\Phi_n(x)$, where $\Phi_n(x)$ is the n -th cyclotomic polynomial), a modulus q , a module rank d , a norm bound β , and the number of samples m . We can now define the MSIS problem formally.

Definition 2.22 ($\text{MSIS}_{q,R,m,d,\beta}$ [LS14]) *Given $\mathbf{a}_1, \dots, \mathbf{a}_m \in R_q^d$ chosen independently from the uniform distribution and forming the columns of a matrix $\mathbf{A} \in R_q^{d \times m}$, find $\mathbf{z} \in R^m$ such that $f_{\mathbf{A}}(\mathbf{z}) := \mathbf{A}\mathbf{z} = \sum_{i=1}^m \mathbf{a}_i \cdot z_i = 0 \pmod{q}$ and $0 < \|\mathbf{z}\| \leq \beta$.*

In simple terms, the MSIS problem requires finding a short nonzero polynomial vector $\mathbf{z}$ with a norm smaller than a given bound β such that multiplied by matrix $\mathbf{A}$ consisting of vectors $\mathbf{a}$, results in zero vector in the ring R_q^d .

SIS and RSIS can be viewed as special cases of MSIS. Specifically, MSIS with module rank $d = 1$ is Ring-SIS. When MSIS is defined over $\mathbb{Z}_q$, it corresponds to the standard SIS.

Unlike MLWE, which is often the basis for encryption and key exchange, MSIS, because it is defined as a search problem [Ajt96, PR06, LS14], is primarily used as a foundation for authentication and integrity primitives.

3 Attacks and Hardness Estimation for LWE and SIS

When designing lattice-based cryptographic schemes, we need to estimate how hard it is to solve the underlying problems (like LWE or SIS) from a computational point of view. The security strength, that is, the required computational effort an attacker would need to break the scheme, can be expressed in bits of security. For example, 64-bit security means that an attack would require at least 2^{64} basic operations (i.e., elementary computational steps). It is important to know whether the chosen parameters are secure enough to withstand known attacks using currently available algorithms, since parameter choices directly impact both the computational hardness and the efficiency of the scheme. For analyzing the security of a specific LWE or SIS instance, we typically attack lattice instances constructed from the corresponding problems. We can model known attacks against lattices using a tool called the Lattice Estimator.

3.1 Lattice Estimator by Albrecht

The Lattice Estimator was developed by Martin Albrecht and his colleagues and was introduced as a Sage module in [APS15]. This tool is widely used in the cryptographic community and in cryptographic research [ABD⁺19, ACD⁺18]. It provides bit-security estimations and allows us to evaluate attack costs individually or for all supported attacks. Currently, the estimator can output the computational complexity for various attacks on LWE and one attack on SIS [Latnd].

With the function `LWE.estimate`, we can evaluate an LWE instance against the most relevant attacks and obtain detailed results grouped by attack. The input and output of the function can be seen in Listing 3.1.1. To initialize an LWE instance, we require several parameters. The parameter n specifies the LWE dimension, q denotes the modulus, Xs is the secret noise distribution and Xe is the error noise distribution. The input and output of the function `SIS.estimate` can be seen in Listing 3.4.1 and 3.4.2. To initialize an SIS instance, several parameters are also required. The parameter n specifies the SIS dimension, q denotes the modulus, `length_bound` is the norm bound and `norm` is chosen norm - the Euclidean or the infinity norm. Every attack outputs specific parameters, which we will discuss further in the corresponding sections. Some output parameters are common to all or most attacks, such as `rop`, `red` and `mem`, which we explain now.

The parameter `rop`, present in every estimate, specifies the total number of word operations (i.e., CPU cycles). This is the most interesting and relevant parameter for

us, as it is nothing else but the bit security, that shows the estimated computational hardness of the problem.

Another key metric in the lattice estimator is `red`, which indicates the cost of the basis reduction process in terms of word operations. Some attacks have their main bottleneck in the amount of memory needed by the attack. In such cases, instead of `red`, the estimator outputs the `mem` parameter, which denotes the total memory usage.

In addition to `LWE.estimate`, the lattice estimator provides a simplified estimation function called `LWE.estimate.rough`. This function allows quick, but rough, security estimations for certain attacks under specific conditions and assumptions. It is useful for getting a fast overview of the hardness of an LWE instance. You can compare the outputs in Listing 3.1.2 to those from the standard estimation function. The security estimations provided by the `LWE.estimate.rough` function are typically lower. This is because `LWE.estimate.rough` relies on several optimistic assumptions, which often lead to more aggressive estimates of the computational cost [Latnd].

```
1  params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
2  ND.UniformMod(3))
3  LWE.estimate(params)
4  # Output
5  # arora-gb      :: rop:  $\approx 2^{40.8}$ , dreg: 3, mem:  $\approx 2^{40.8}$ , t:
6  1, m:  $\approx 2^{20.4}$ , tag: arora-gb,  $\mathfrak{s}$ : 1,  $\zeta$ : 0
7  # bkw          :: rop:  $\approx 2^{75.7}$ , m:  $\approx 2^{65.5}$ , mem:  $\approx 2^{66.5}$ ,
8  b: 9, t1: 3, t2: 9,  $\ell$ : 8, #cod: 141, #top: 1, #test: 31, tag
9  : coded-bkw
10 # usvp         :: rop:  $\approx 2^{86.4}$ , red:  $\approx 2^{86.4}$ ,  $\delta$ : 1.006187,
11  $\beta$ : 205, d: 378, tag: usvp
12 # bdd          :: rop:  $\approx 2^{82.5}$ , red:  $\approx 2^{80.8}$ , svp:  $\approx$ 
13  $2^{82.0}$ ,  $\beta$ : 184,  $\eta$ : 216, d: 380, tag: bdd
14 # dual         :: rop:  $\approx 2^{90.8}$ , mem:  $\approx 2^{54.7}$ , m: 204,  $\beta$ :
15 217, d: 404,  $\mathfrak{s}$ : 1, tag: dual
16 # dual_hybrid  :: rop:  $\approx 2^{82.8}$ , red:  $\approx 2^{82.8}$ , guess:  $\approx$ 
17  $2^{77.6}$ ,  $\beta$ : 188, p: 4,  $\zeta$ : 10, t: 20,  $\beta'$ : 199, N:  $\approx 2^{41.1}$ ,
18 m: 200
```

Listing 3.1.1: Example output of the `LWE.estimate` function.

As we have already mentioned, along with the functions that evaluate several attacks at once, the lattice estimator also provides individual functions to simulate each attack separately. These per-attack functions give us finer control over modelling choices and other parameters. A detailed tuning of these options is not in the scope of this thesis. Therefore, we use the default configuration of the estimator.

```

1  params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
    ND.UniformMod(3))
2  LWE.estimate.rough(params)
3  # Output
4  # usvp          :: rop:  $\approx 2^{59.9}$ , red:  $\approx 2^{59.9}$ ,  $\delta$ : 1.006187,
     $\beta$ : 205,  $d$ : 378, tag: usvp
5  # dual_hybrid  :: rop:  $\approx 2^{60.8}$ , red:  $\approx 2^{60.7}$ , guess:  $\approx$ 
     $2^{57.4}$ ,  $\beta$ : 199,  $p$ : 4,  $\zeta$ : 0,  $t$ : 20,  $\beta'$ : 199,  $N$ :  $\approx 2^{43.8}$ ,  $m$ :
    200
6  # arora-gb      :: rop:  $\approx 2^{40.8}$ , dreg: 3, mem:  $\approx 2^{40.8}$ ,  $t$ :
    1,  $m$ :  $\approx 2^{20.4}$ 

```

Listing 3.1.2: Example output of the `LWE.estimate.rough` function.

3.2 Primal Attacks on LWE

We begin with attacks that can be classified into one family known as primal attacks. These attacks attack the original (i.e., primal) lattice to find secrets directly. In essence, all these attacks follow the same pattern and have the same core idea. First, we construct a lattice from the given LWE instance. Then we apply a basis reduction algorithm to obtain the reduced basis. Finally, we use this basis to recover the secret. The differences between the primal attack variants lie in how the lattice is constructed and which success condition is used. We can now proceed to model each primal attack individually, beginning with the primal uSVP attack.

Primal uSVP attack. The basic idea behind the primal uSVP attack follows the standard primal pattern [AFG13, ZZW22]. We construct a lattice based on the given LWE samples (using a special way to build it), reduce its basis by a lattice basis reduction algorithm. Success in primal uSVP attack means solving a special variant of SVP problem after obtaining the reduced basis - unique Shortest Vector Problem. Unlike the standard SVP, where several shortest can exist, in uSVP the shortest vector must be unique, up to multiplication by ± 1 . If the obtained basis was good enough, then the target vector, that will help us to reveal the secret, will stand out as the only shortest vector.

In the lattice estimator output in Listing 3.2.1 besides the common parameters we can see other specific parameters. To quantify the strength of lattice reduction algorithm to measure how good a lattice basis is after reduction, we use a metric called the root Hermite factor δ , $1 \leq \delta$ [ABF⁺20]. The closer the root Hermite factor is to 1, the better the basis we achieved. In practice, $\delta = 1.005$ is considered pretty good, and $\delta = 1.02$ weak. In the output δ is the targeted root Hermite factor by the lattice reduction. The parameter β is the BKZ block size and is used by a lattice reduction algorithm. The larger β , the higher the computational cost, but the better the reduction, and, as consequence, the smaller δ we get. The constructed lattice dimension we read from the parameter d .

```

1  params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
    ND.UniformMod(3))
2  LWE.primal_usvp(params)
3  # Output
4  # rop:  $\approx 2^{86.4}$ , red:  $\approx 2^{86.4}$ ,  $\delta$ : 1.006187,  $\beta$ : 205,  $d$ : 378,
    tag: usvp

```

Listing 3.2.1: Example output of the `LWE.primal_usvp` function.

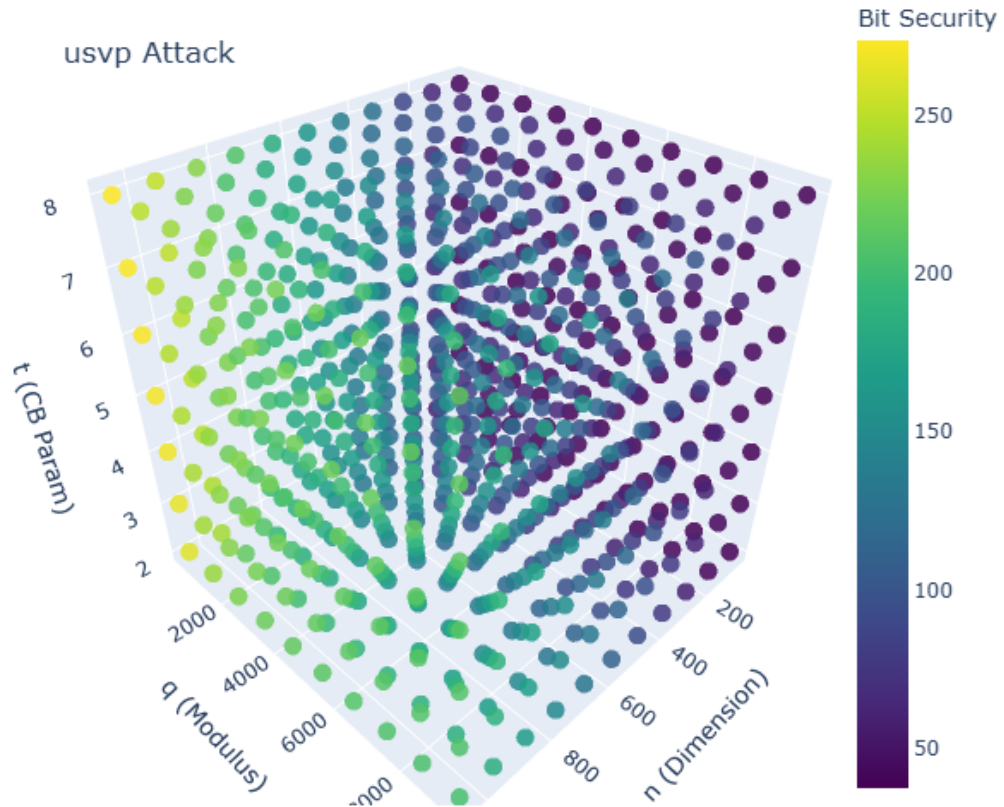


Figure 3.1: Primal uSVP attack - estimated bit security over a 3D grid

From the data we collected, we can recognize how different parameters impact the attack. The LWE dimension has the main impact on the primal uSVP attack. Increasing the dimension increases the hardness of the problem. On the other hand, increasing the modulus q facilitates the attack.

Primal BDD attack. Another representative of the primal attack family is the primal BDD attack. BDD stands for the Bounded Distance Decoding problem [LN13]. We omit the formal definition of it. In essence, given a lattice and a target vector, we must find the lattice point closest to the target, under the condition that the target vector actually exists within a fixed radius of some lattice point.

Like other primal approaches, the attacker works in the original lattice built from the given LWE samples. We construct a special lattice, but in a different way than in primal uSVP, and apply basis reduction. Then, on the reduced basis, we perform addition computations to solve the BDD problem with a right-hand side of the LWE as the target vector. The success condition of the primal BDD attack is finding the correct lattice point, so we get enough information to recover the secret.

Among already known parameters, such as `rop`, `red`, β and d we have also new output parameters in Listing 3.2.2. The parameter η specifies the dimension of the final BDD problem and `svp` is the computational cost of the SVP step, because in practice, solving the BDD on a reduced basis is implemented using a shortest-vector algorithm.

```

1      params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
      ND.UniformMod(3))
2      LWE.primal_bdd(params)
3      # Output
4      # rop:  $\approx 2^{82.5}$ , red:  $\approx 2^{80.8}$ , svp:  $\approx 2^{82.0}$ ,  $\beta$ : 184,  $\eta$ :
      216, d: 380, tag: bdd

```

Listing 3.2.2: Example output of the `LWE.primal_bdd` function.

From the plot, we can see that the parameter impact on the hardness of the problem is similar to that for the primal uSVP attack.

Primal Hybrid attack. While both primal uSVP and primal BDD attacks rely purely on lattice reduction algorithms, the following approach, known as the primal hybrid attack, combines lattice reduction with a probabilistic component. The primal hybrid attack adds a guessing step to a standard primal attack strategy. The intuition is that, before the reduction step, we make a smart guess about a small part of the secret [Wun18]. We use this guess to simplify the initial instance (e.g., reduce the dimension or noise of the given LWE samples). Thereafter, we apply lattice reduction to improve the lattice basis and then proceed as in the primal BDD attack. If the guess is correct, the short vector stands out, and we can obtain information about the secret. If the secret vector is initially sparse or small, guessing becomes feasible. There are many

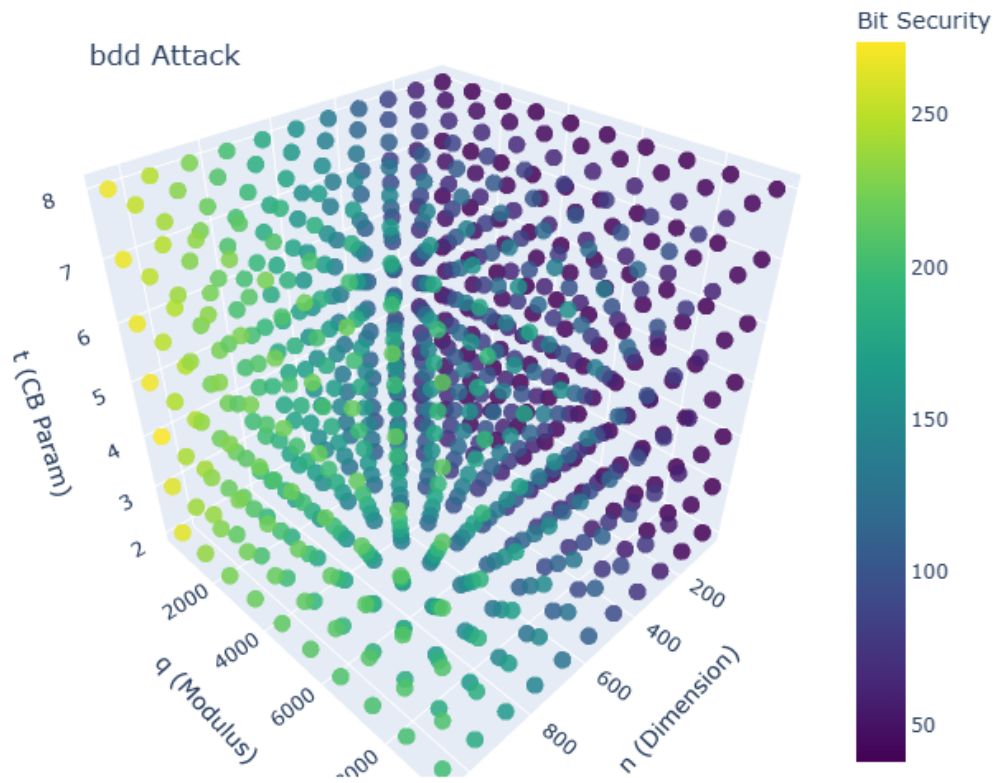


Figure 3.2: Primal BDD attack - estimated bit security over a 3D grid

different ways of guessing a part of the secret, leading to multiple variants of the attack [ACW19, Wun19].

In addition to the parameters already introduced in previous sections (`rop`, `red`, β , `svp`, η , d), in the lattice estimator output in Listing 3.2.3 we see new parameters. The parameter ζ is the number of guessed coordinates in the target vector, and the parameter $|S|$ is the size of the guessing search space. There is a trade-off between these two parameters: the more coordinates we guess, the bigger the search space becomes. Larger ζ shrinks the reduced basis but grows S exponentially.

Another two parameters are `prob` and $\mathfrak{e}$ (i.e., repetitions). The first stands for the probability of successful guessing, and the second lets us know how often the hybrid attack was repeated.

```

1  params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
    ND.UniformMod(3))
2  LWE.primal_hybrid(params)
3  # Output
4  # rop:  $\approx 2^{114.8}$ , red:  $\approx 2^{113.9}$ , svp:  $\approx 2^{113.7}$ ,  $\beta$ : 205,  $\eta$ :
    2,  $\zeta$ : 87,  $|S|$ :  $\approx 2^{137.9}$ ,  $d$ : 311, prob:  $\approx 2^{-26.0}$ ,  $\mathfrak{e}$ :  $\approx$ 
     $2^{28.2}$ , tag: hybrid

```

Listing 3.2.3: Example output of the `LWE.primal_hybrid` function.

3.3 Other Attacks on LWE

Having presented the primal attack family, we will discuss in this section other attacks and approaches for solving the LWE problem and estimating its hardness. Unlike primal attacks, these methods rely only partly on lattice reduction techniques, or do not involve a lattice reduction step at all.

Dual attack. We begin with the dual attacks [MR09]. Dual attacks are performed on the correspondingly built dual lattice, in contrast to primal attacks, which are performed directly on the original lattice. Another difference is that the dual attack is a distinguishing attack and tries to solve the Decision-LWE problem, not the search variant.

The basis of the dual lattice has one property useful for an attacker: its length is inversely proportional to the length of the original basis. Dual attacks exploit the fact that short vectors can be easier to obtain in the dual lattice, even if the primal lattice basis has a large norm.

Let us briefly outline the intuition behind the attack. First, we build the corresponding dual lattice. In the next step we apply lattice reduction to obtain a better basis and then search for a short dual vector. This short vector has the property that, in combination with the given samples, we detect a small statistical bias if the given samples are actual

LWE samples. If the samples are uniform, we see no bias. After obtaining this short vector, we run a statistical test on the samples. A faster variant of the dual lattice attack, which improves how we find and use the short dual vector, is introduced in the [GJ21].

In the output of the estimation in Listing 3.3.1, we have the `mem` parameter, indicating that total memory usage is more relevant here. The parameters `rop`, β , d and $\mathfrak{s}$ have already been defined in previous sections. Because the statistical test on the samples is probabilistic, we also see a new parameter m , which denotes the required number of samples to distinguish the correct solution with high probability. The more samples we have, the higher the probability of a correct solution.

```

1      params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
      ND.UniformMod(3))
2      LWE.dual(params)
3      # Output
4      # rop:  $\approx 2^{90.8}$ , mem:  $\approx 2^{54.7}$ ,  $m$ : 204,  $\beta$ : 217,  $d$ : 404,  $\mathfrak{s}$ : 1,
      tag: dual

```

Listing 3.3.1: Example output of the `LWE.dual` function.

As we can recognize, the dimension of the LWE problem and the modulus play the crucial role in the hardness of the attack.

Hybrid dual attack. Exactly as the primal hybrid attack, in the dual hybrid attack [BLL⁺22] we add a probabilistic component, while the standard dual attack is deterministic, except for the statistical test. We guess a small part of the secret first, which simplifies the LWE instance. Typically, this allows us to increase the bias used in dual attack strategy. Then we build the corresponding dual lattice and apply the lattice basis reduction algorithm. Thereafter we find a short dual vector, and finally run the statistical test on the provided samples to distinguish whether the given samples are LWE samples or uniformly random ones. The hybrid dual attack, similar to the primal hybrid attack, works well on non-sparse secrets because the guess becomes more effective.

There are also variants of the dual hybrid attacks that improve the efficiency of the attack using some additional techniques [CHHS19, BLLW22]. Some dual hybrid attack variants apply an additional reduction on the full dual lattice before the guessing step [EJK20].

In the estimation output in Listing 3.3.2, among already introduced output parameters, such as `rop`, `red`, β , ζ and m , we have some specific parameters. The parameter `guess` denotes the cost of the guessing step. While β is the block size of the reduction used by a lattice reduction algorithm before the guessing step, the parameter β' is the block size used after the guessing step. The parameter t is the number of secret coordinates to guess mod 2 and the parameter N denotes the number of candidates tested after guessing. The parameter p is the FFT modulus used for the additional computations in the guessing stage.

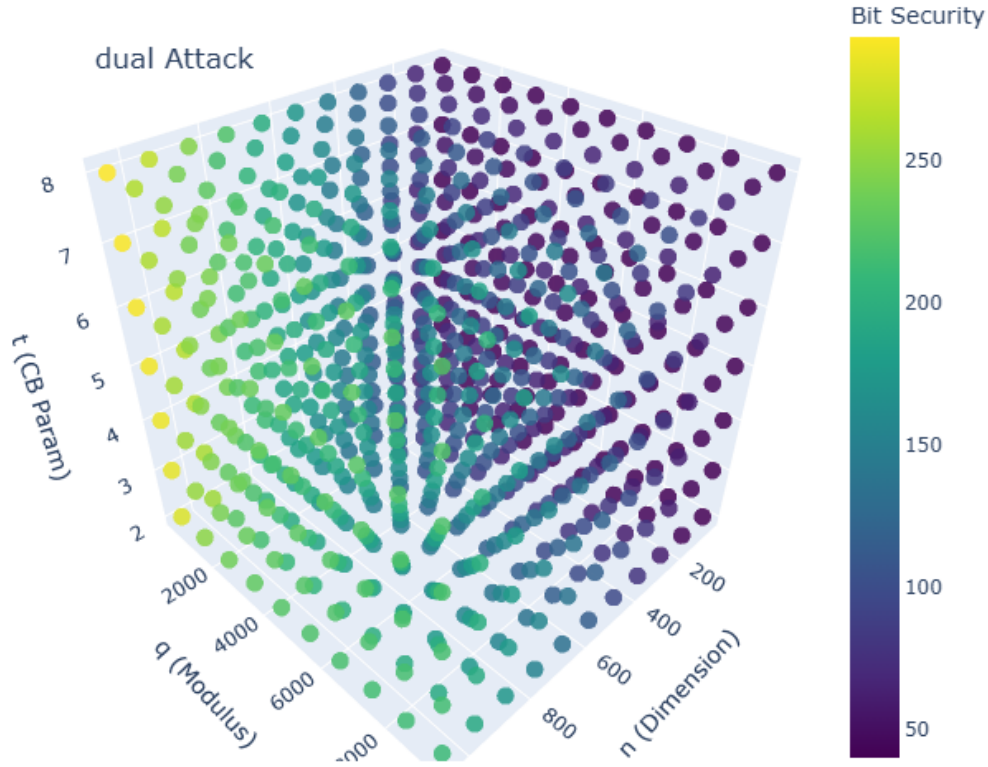


Figure 3.3: Dual attack - estimated bit security over a 3D grid

```

1  params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
   ND.UniformMod(3))
2  LWE.dual_hybrid(params)
3  # Output
4  # rop:  $\approx 2^{82.8}$ , red:  $\approx 2^{82.8}$ , guess:  $\approx 2^{77.6}$ ,  $\beta$ : 188, p:
   4,  $\zeta$ : 10,  $t$ : 20,  $\beta'$ : 199,  $N$ :  $\approx 2^{41.1}$ ,  $m$ : 200

```

Listing 3.3.2: Example output of the LWE.dual_hybrid function.

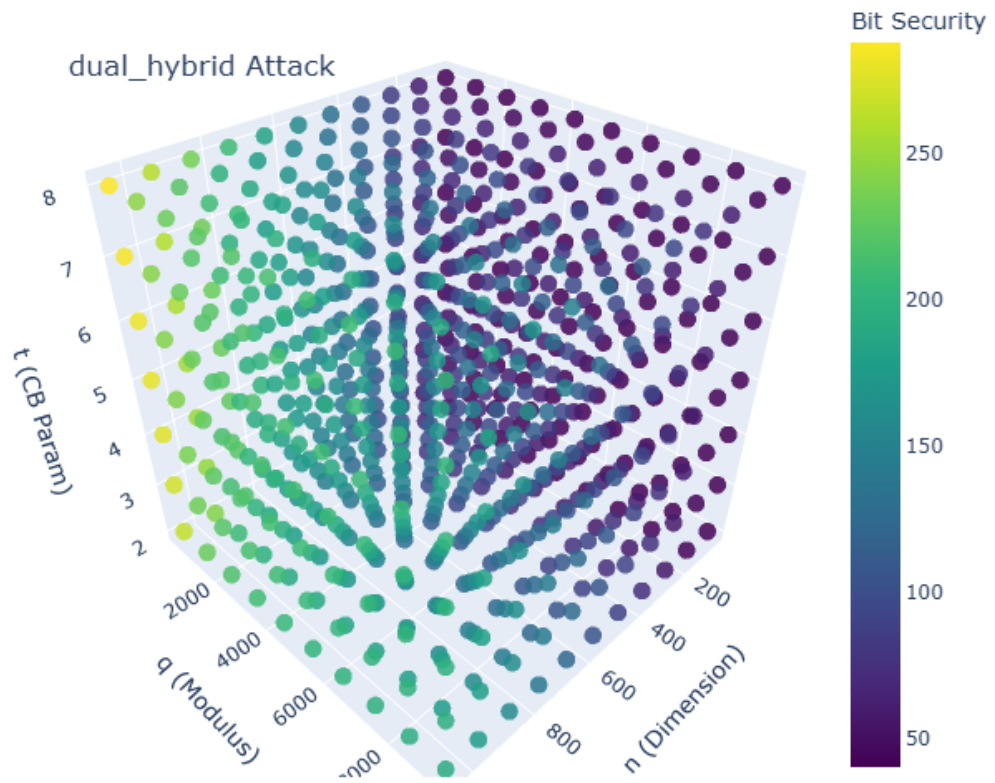


Figure 3.4: Dual hybrid attack - estimated bit security over a 3D grid

As we can see, the dimension of the LWE problem as well as the modulus do not significantly affect the hardness of the attack. What has a greater impact is the alphabet of the secret and the range of the error term.

Arora-GB Attack. There are some attacks that do not build any special lattice and do not rely on any lattice reduction algorithms, unlike primal or dual attacks. The Arora-GB attack is a completely algebraic method to solve the LWE problem. It works directly with the LWE equations, exploiting the boundedness of the error term $|e| \leq t$, $t \in \mathbb{Z}$ and the fact that it is relatively small. The main idea is to use a special polynomial that has the possible error values as roots, so the error disappears completely and we can solve a polynomial (i.e., algebraic) system to obtain the secret.

In essence, we take many LWE samples and, for each sample, apply a specially chosen polynomial to the equation. Let us explain it on an example. If the error $e \in \{-1, 0, 1\}$, we can use the polynomial $P(e) = e(e-1)(e+1)$, which is zero for all valid error values. Substituting e with the sample expression turns each noisy sample into a polynomial equation. If we have enough LWE samples, we have a large system that we can then solve with a method called a Gröbner basis method [AG11]. It is a pure equation-solving approach that solves a modified algebraic system.

The attack requires a significant amount of memory, especially when the polynomial system becomes too big. That is why, in addition to `rop`, we see the `mem` parameter in the output in Listing 3.3.3. Another parameter, `dreg`, denotes the highest total degree of polynomials reached during solving the obtained algebraic system. The parameter t is the number of considered polynomials of degree $2t + 1$, and the number of consumed LWE samples is given by the parameter m .

```

1      params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
      ND.UniformMod(3))
2      LWE.arora_gb(params)
3      # Output
4      # rop: ≈ 2^40.8, dreg: 3, mem: ≈ 2^40.8, t: 1, m: ≈ 2^20.4,
      tag: arora-gb

```

Listing 3.3.3: Example output of the `LWE.arora_gb` function.

We can see that the LWE dimension and the modulus have almost no influence on the estimated computational hardness of the problem. The main impact comes from the possible values of the error term e . The more possible values there are, the more computational effort the attack requires.

BKW Attack. Another attack that does not use lattice reduction is the BKW attack [BKW03]. Unlike the Arora-GB attack that is an algebraic method, the BKW attack is a combinatorial method. The idea of the attack is to combine LWE samples in a clever way and gradually eliminate variables from the LWE equations, until the dimension of the sample system is reduced enough to solve it directly by brute force or by guessing.

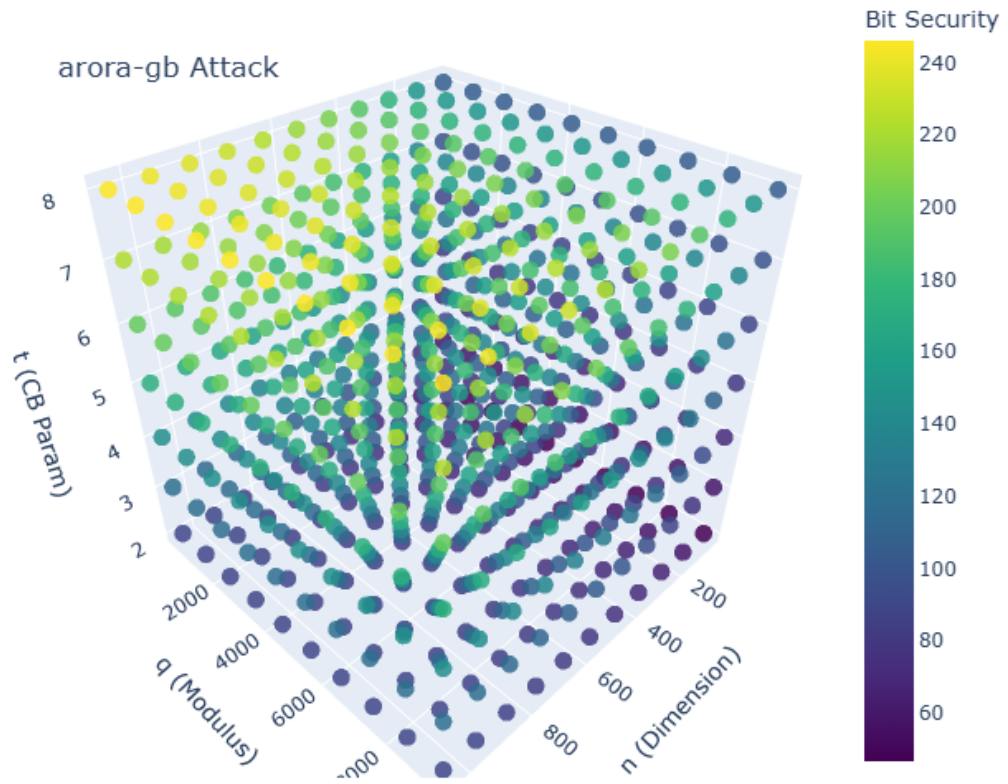


Figure 3.5: Arora-GB attack - estimated bit security over a 3D grid

In essence, we start with a large set of LWE samples and repeatedly group samples with matching patterns. Then, using addition, subtraction or XOR operations, we cancel out some secret components so they disappear from the equations. This reduces the effective problem size (i.e., the dimension) but increases the noise. After enough eliminations, we have short equations in a few variables and finish by a small brute force, because the problem becomes small enough.

The BKW attack has variants that improve the attack [KF15]. For example, some BKW variants use error-correcting codes during the combining step [GJS15]. This technique allows us to get cleaner cancellations and reduce the noise growth. In the lattice estimator output in Listing 3.3.4 this appears with parameters such as `#cod` - the number of coding steps, and `t2` - the number of coded BKW steps. The parameter `t1` denotes the number of normal BKW steps. The number of guessing steps is given by the parameter `#top`. The parameter `m` is the number of LWE samples required for the attack, and the parameter `b` is the table size for BKW steps in q^b . The parameters for the final part of the BKW attack, `ℓ` and `#test`, denote the table size for hypothesis testing in $q^{\ell+1}$ and the number of coordinates to do hypothesis testing, respectively.

```

1  params = LWE.Parameters(n=200, q=127, Xs=ND.UniformMod(3), Xe=
    ND.UniformMod(3))
2  LWE.coded_bkw(params)
3  # Output
4  # rop: ≈ 2^75.7, m: ≈ 2^65.5, mem: ≈ 2^66.5, b: 9, t1: 3, t2:
    9, ℓ: 8, #cod: 141, #top: 1, #test: 31, tag: coded-bkw

```

Listing 3.3.4: Example output of the `LWE.coded_bkw` function.

3.4 Attacks on SIS

We have discussed all attacks on LWE that are supported by the lattice estimator. Now we can move to the attack on the SIS problem. The estimator by Albrecht provides only limited functionality for attack modelling on SIS. The only supported attack is the so-called BKZ attack. BKZ lattice reduction algorithm can be used to solve the SIS problem [QLZ⁺23].

The intuition is similar to the primal uSVP attack on LWE. First we build the lattice from the matrix $\mathbf{A}$ that encodes the given SIS instance, apply the lattice reduction algorithm BKZ to get a better basis. Then if the reduction is good, the short non-zero vector satisfying $\mathbf{A}\mathbf{z} = 0 \pmod{q}$ and the norm bound constraint will stand out in the reduced basis. Success depends on the reduction quality and on the required tightness of the norm bound for the target vector $\mathbf{z}$.

The estimator distinguishes two cases in terms of the chosen norm bound: the Euclidean norm and the infinity norm. The SIS problem estimation is provided in Listing 3.4.1. The parameters given there such as `rop`, `red`, β , δ and d were already presented.

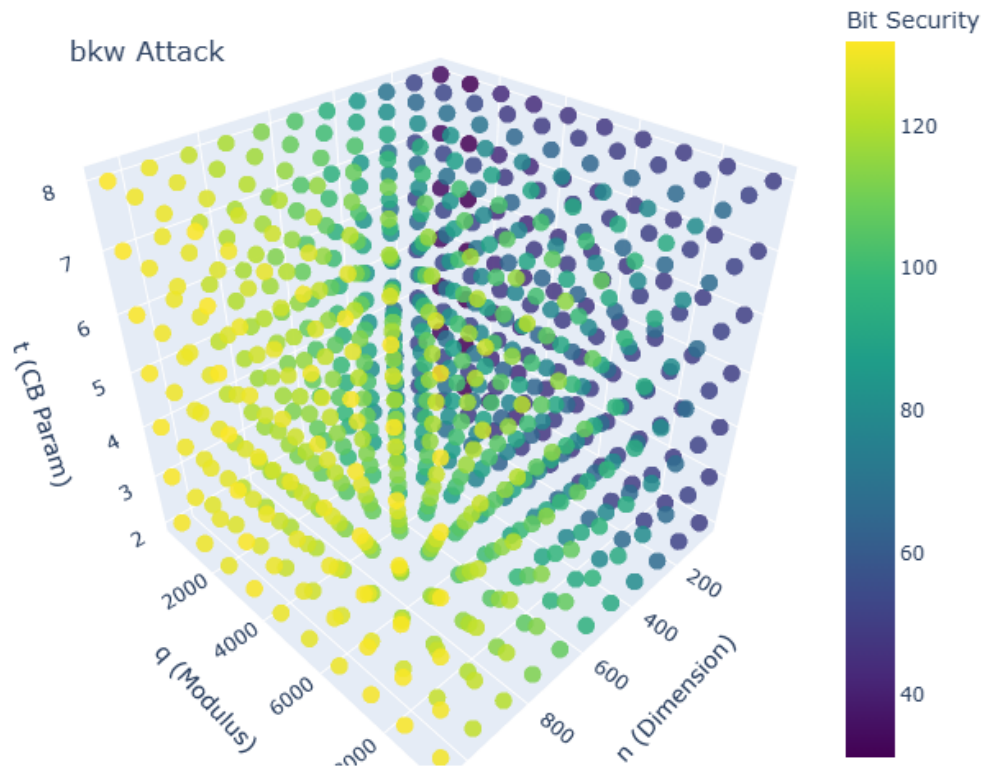


Figure 3.6: BKW attack - estimated bit security over a 3D grid

In the estimator output for the infinity norm in Listing 3.4.2 we see some new parameters. The parameter ζ is the number of coefficients of the solution vector that are set to 0 in order to reduce the effective search dimension. In the infinity norm bound case we need an additional step for an extra procedure called sieving. In the sieving step, we iteratively combine and replace lattice vectors, keeping shorter ones and discarding longer ones, until a sufficiently short vector is found. The parameter `sieve` is the cost of this additional computation. The parameter η is the dimension of the final sieving step. The estimator also allows a probabilistic search over candidate short vectors after reduction, since under the infinity norm (coordinate-wise) bound there are may be many candidates. In the Euclidean norm case there are usually far fewer candidates, so no extra search is modeled. The success probability of this probabilistic step is given by the parameter `prob`, and the number of repetitions is indicated by the parameter L .

```

1  params = SIS.Parameters(n=113, q=2048, length_bound=512, norm
2  =2)
3  SIS.estimate(params)
4  # Output
# lattice  :: rop:  $\approx 2^{47.0}$ , red:  $\approx 2^{47.0}$ ,  $\delta$ : 1.011391,  $\beta$ :
61, d: 276, tag: euclidean

```

Listing 3.4.1: Example output of the `SIS.estimate` function with Euclidean norm.

```

1  params = SIS.Parameters(n=113, q=2048, length_bound=512, norm=
2  oo)
3  SIS.estimate(params)
4  # Output
# lattice  :: rop:  $\approx 2^{40.7}$ , red:  $\approx 2^{39.7}$ , sieve:  $\approx 2^{39.7}$ ,
 $\beta$ : 40,  $\eta$ : 57,  $\zeta$ : 113, d: 137, prob: 1,  $\mathfrak{s}$ : 1, tag: infinity

```

Listing 3.4.2: Example output of the `SIS.estimate` function with infinity norm.

4 Development of the Parameter Selection Program

4.1 Main Goal and Motivation

In the previous sections we have provided the required mathematical background by introducing formal definitions of the primitives. We have also formally defined the essential lattice problems used in lattice-based cryptography and have given their intuition. Thereafter we have explained the intuition behind the well-known attacks on LWE and SIS and introduced how to estimate hardness with the lattice estimator by Albrecht [APS15]. In essence, the estimator answers: given concrete parameters, how hard is the problem to break? However, in practice, when deploying a cryptographic scheme we often need the inverse: given a target bit security and specific constraints, which concrete parameter values should we choose so that the problem instance remains secure and computation is efficient? Doing this manually, probing many candidates, checking several attacks each time, and reapplying constraints, costs both computation and human time and is easy to get wrong. This motivates an automated approach that systematically computes valid parameters while keeping them efficient.

We have developed an automated program that uses the lattice estimator to evaluate hardness but works in the opposite direction. Given a required bit security and user-specified constraints, it computes concrete parameters for the corresponding problem that meet the target security against standard attacks supported by the estimator and satisfy the given constraints, trying to optimize parameter values to remain efficient. Our goal was a program, that respects real-world cryptographic design constraints, supports very large domains (i.e. modulus q up to 2^{500}), balances security and efficiency, and provides clear inputs and outputs so results are reproducible and auditable.

4.2 Basic Functionality and Design Decisions

Now we describe the functionality of the program in detail, what features are provided and how to use it. Furthermore, we examine the difficulties and challenges encountered during the realization process. Our program is a Python CLI script with many optional features, which we will discuss one by one later with short usage example and the design decisions behind them. The tool currently supports LWE and SIS and their ring and module based variants, which we introduced in the "Mathematical Foundations" section. We begin with the basic usage without any optional functionality and after that will move on to additional features.

Using one large single monolithic script for all supported problems, would result in excessive conditional logic; splitting the functionality into smaller, problem-specific scripts produces cleaner code but requires an additional coordinator or wrapper. To make the usage simple, despite of multiple different problems with different parameter sets per problem, we wrapped the problem selection program in a single `finder.py` script. This provides us one single entry point, which takes with the first positional argument the problem name (LWE, RLWE, MLWE, SIS, RSIS, MSIS), forwards the rest of the CLI to the corresponding problem-specific script. Our program also keeps resource usage predictable and lower memory footprint, which is useful when users run many batches. In the example input of the basic usage for the LWE problem in Listing 4.2.1, we see the name of the problem in the first positional argument - here LWE. The flag `-bit <integer>` sets the minimum target bit security. Every supported attack by the estimator's function `LWE.estimate` must meet at least the entered bit security. The next flags `-n <integer>`, `-q <integer>`, `-t {<integer> | <real>}` just name the required parameters for an LWE instance. The names of the parameters we will need later. `-n` is the LWE dimension, `-q` denotes the modulus and `-t` is the parameter of the distribution, from which the error term is drawn. By default we use Centered Binomial (CB) for the error distribution. In this case, the standard deviation is $\sigma = \sqrt{\frac{t}{2}}$. In the same Listing 4.2.1, we see the inputs for RLWE and MLWE. The basic input for RLWE is the same as for the LWE problem. However, in the RLWE script it is assumed that the dimension of the problem must be a power of two. The MLWE input includes new parameters. The flag `-d` is the rank of the module and `-N` denotes the total dimension of the corresponding LWE instance. To be able to estimate the MLWE problem with the lattice estimator, we transform the module rank and the MLWE dimension to the total LWE dimension as $N = n \cdot d$. The same technique was used by Lyubashevsky in [LNP22].

The example input of the basic usage for the SIS problem and its variants is shown in Listing 4.2.2. The flags `-n <integer>`, `-q <integer>`, `-len <integer>`, `-beta {0 | 2}` name the required parameters for an SIS instance. `-len` is the length bound and `-beta` is the chosen norm - 0 denotes the infinity norm and 2 denotes the Euclidean norm.

All possible parameters for our program are listed and briefly described in Table 4.1.

```

1  python3 finder.py LWE -bit 40 -n n -q q -t t
2  # Output
3
4  python3 finder.py RLWE -bit 40 -n n -q q -t t
5  # Output
6
7  python3 finder.py MLWE -bit 40 -N N -n n -d d -q q -t t
8  # Output

```

Listing 4.2.1: Basic program inputs for the LWE, RLWE and MLWE problems.

Flag / Arg	Description	Type / Values	Default
PROBLEM	Problem name (finder entry)	LWE RLWE MLWE SIS RSIS MSIS	–
-bit	Target bit security (minimum)	integer	(required)
-n	Name of dimension variable	string (e.g. <code>n</code>)	(required)
-q	Name of modulus variable	string (e.g. <code>q</code>)	(required)
-t	Name of error-param variable	string (e.g. <code>t</code>)	(required for LWE)
-N	Total dimension (MLWE/MSIS)	integer	(required for MLWE and MSIS)
-d	Module rank (MLWE/MSIS)	integer	(required for MLWE and MSIS)
-len	Length bound	integer	(required for SIS)
-beta	Chosen norm	integer 0 or 2	(required for SIS)
-m	Length of SIS input	integer	set by the estimator
-enable-refine	Enable post-processing refinement	flag	off
-bit-sec-margin	Refinement security margin (bits)	integer	10
-c "..."	Constraint (repeatable)	Z3 expr; supports linear operators and selected non- linear constraints.	–
-v "type x = expr"	Define custom var (repeatable)	<code>type=int real</code>	–
-opt "..."	Add optimization objective (minimize)	var or expr	–
-opt-priority	Optimization mode	<code>lex pareto box</code>	lex
-enable-big	Big-number mode (large domains)	flag	off
-error-distribution	Error distribution	Look the "Noise distributions" paragraph.	cbinomial
-secret-distribution	Secret distribution	Look the "Noise distributions" paragraph.	cbinomial
-s	Secret distribution parameter	integer or real (de- pends on the dis- tribution)	(distribution-specific)

Table 4.1: Command-line parameters and flags (overview). **PROBLEM** is the first positional arg to **finder**.

```
1 python3 finder.py SIS -bit 40 -n n -q q -len L  
   -beta 2  
2 # Output  
3  
4 python3 finder.py RSIS -bit 40 -n n -q q -len L  
   -beta 2  
5 # Output  
6  
7 python3 finder.py RSIS -bit 40 -N N -n n -d d -q q -len L  
   -beta 2  
8 # Output
```

Listing 4.2.2: Basic program inputs for the SIS, RSIS and MSIS problems.

Z3 Solver. Even with just three core variables, finding a good parameter set is not trivial. The reason is that each variable affects hardness in a different way, because attacks react differently to changes in the parameters. Once we add user-defined variables and varying constraints, which makes the relations between parameters complex and tangled, the search space becomes huge and the interactions become messy. Brute force is not realistic, because manual exploration may miss valid solutions or require excessive computational effort. Handling this problem manually is challenging, and developing a complete constraint engine would represent an independent project. For this reason we have decided to use a Satisfiability Modulo Theories (SMT) solver from Microsoft called Z3, or the Z3 Theorem Prover [DMB08]. Modelling our task as an SMT problem let us focus on value optimization to get an optimal parameter set for the problem instance, the interpretation of the estimator outputs, and how to shape the search.

The solver acts as a filter. Instead of trying every possible parameter set, first we declare our variables in Z3 and state constraints over them. Thereafter, we let the solver find values that satisfy all constraints, and then we evaluate only those candidates with the lattice estimator to check whether the target bit security is achieved. This lets us speed up our computation.

Another reason why we have chosen Z3 is that it provides a declarative style for our program. We can specify what must hold instead of coding a big amount of imperative checks, which is simpler to maintain and to justify. The solver also provides reproducibility. We can be sure that the same constraints lead to the same solver state. Our program remains completely deterministic and our results remain auditable. Also, the SMT solver allows correct integer and real handling and support for very large domains.

Z3 provides many built-in features which we wanted to realize in our program initially. We will talk about these features further in this section. However, the solver from Microsoft does not support non-linear constraints to the extent that we need. Nevertheless, we introduce our implemented methods to address that issue. During development we have tested another SMT solver called cvc5 [BBB⁺22], which promised us wider support for non-linear algebra. The handling of non-linear constraints was also not enough for

our purposes. Furthermore, it does not provide as many features useful to us as Z3 and is slower than its rival. That is why we settled on this solution.

Search and computation strategy. With SMT solver explained, we can introduce our search strategy, which we call windowed search. The thing is that even after Z3 filters out impossible candidates, calling `LWE.estimate()` or `SIS.estimate()` functions is very expensive, especially for larger dimensions. If we tried to evaluate every satisfying parameter set inside huge domains, we might get slightly more precise values for the parameters, but the computational cost would grow enormously. That means that the problem cannot be solved by trivial search. We need a way to check where it matters, progress quickly, and minimize estimator calls.

To achieve this goal, instead of searching the full domain at once, we bound each required parameter by a relatively small interval (i.e., a window). Formally, intervals for the LWE problem are defined as

$$n \in [n_{lb}, n_{lb} + c_n \Delta_n), \quad q \in [q_{lb}, q_{lb} + c_q \Delta_q), \quad t \in [t_{lb}, t_{lb} + c_t \Delta_t),$$

where n_{lb}, q_{lb}, t_{lb} are lower bounds for the current attempt; $\Delta_n, \Delta_q, \Delta_t$ are predefined constant steps for each parameter; and c_n, c_q, c_t are dynamically adaptive coefficients for the steps. For the SIS problems intervals for the dimension and the modulus are the same. The distribution parameter is not needed.

With intervals, we limit the search space for our SMT solver. We let Z3 find a candidate that satisfies all structural constraints only within these windows. Thereafter, we call the estimator on that candidate. This avoids evaluating infeasible parameter sets and significantly reduces the number of estimator calls, and allows to find an optimized solution more efficiently.

As we have mentioned in the "Attacks and Hardness Estimation for LWE and SIS" section, different parameters influence specific attacks. In essence, the LWE dimension impacts the lattice reduction step in such attacks as primal uSVP, primal BDD, dual, dual hybrid and BKW. The modulus primarily influences the BKW attack, and the parameter t for the error term distribution is the main lever for the Arora-GB attack. In the SIS case, increase of the dimension or the modulus makes the instance harder to solve, while increase of the length bound makes the instance easier. The discovered correlations allow us, once the estimator outputs the security of each attack, to selectively adjust only the relevant parameter associated with the attack that breaks our problem instance more effectively than the required hardness.

If Z3 does not find any solution, that meets all specified constraints, we expand the search space for the solver, expanding the interval. If we cannot find an appropriate parameter set twice in a row, we double the expansion factor and expand windows faster to reach the right search space much faster. Once we find a feasible interval, we tighten the factor and the window again.

The example flow of the program for the LWE problem can be seen in Listing 4.2.3. Other problems follow the same strategy. At the end, we get an appropriate solution based on the required hardness and constraints. The final result is shown in Listing 4.2.4.

In lines 1-4 the attempt number and current windows for the parameters are shown. In line 5 we see the solution found by Z3. After every estimation, the estimator's output is listed, which we omit here. For better understanding, we then output the extracted bit security per attack. Lines 8-10 show which parameters will be increased in the next iteration. From line 12 we see the beginning of the second attempt.

```
1      Attempt 1:
2      n ∈ [41, 71)
3      q ∈ [300, 500)
4      t ∈ [1, 2)
5      Z3 solution found: n=41, q=300, t=1
6      # Estimator verbose output
7      Bit security: {'usvp': 36.4, 'dual': 40.7, 'dual_hybrid':
8                    39.2, 'bkw': 26.4, 'arora-gb': 28.8, 'bdd': 38.0}
9      Increasing n
10     Increasing q
11     Increasing t
12
13     Attempt 2:
14     n ∈ [71, 101)
15     q ∈ [500, 700)
16     t ∈ [2, 3)
17     ...
```

Listing 4.2.3: Program output for the LWE problem.

```
1      ...
2      All attacks passed threshold!
3      n = 101
4      q = 700
5      t = 2
```

Listing 4.2.4: Program output for the LWE problem.

4.3 Advanced Functionality

Having introduced the basic workflow, we can now turn to the advanced features. In our program, users can add constraints, introduce custom variables, set optimization goals, and others while keeping runs reproducible and readable. For each feature we explain why it is needed, what it does, how to use it, and the design decisions and trade-offs that arose during the implementation.

Constraints. In practice we often need to restrict parameters directly: even dimension, lower bounds on modulus, relations between different parameters, etc. To provide flexibility and ease of use, we allow stating constraints directly on the CLI. The flag `-c` lets us add constraint expressions as strings over the defined in the input variables (the order of definition and defining constraints does not matter). We may combine variables and numeric constants freely and pass multiple `-c` flags. The example of constraint definition can be seen in Listing 4.3.1.

Supported operators include: `/ % * + - < <= == >= > **`, where the operator `**` denotes the power, which restricted to numeric real constants. With `**` we can define roots constraints as well. For example, the square root of the parameter n can be defined with `n ** (1 / 2)`, the cube root can be defined with `n ** (1 / 3)` and so on. The mixed fractions like $(3/2)$ are also supported.

As we mentioned above implemented constraint input with Z3. Namely, we built a CLI Z3 layer: we parse each constraint string, bind names to the corresponding variables, and hand the constraint to Z3.

```

1 python3 finder.py LWE -bit 40 -n n -q q -t t -c "n % 2 == 0"
2 -c "q >= n ** (5 / 3)" -c "(n / 20) ** 2 / (q / 10) > 20"
   # Output

```

Listing 4.3.1: Constraints input for the LWE problem.

Constraints: $n \bmod 2 = 0$, $q \geq n^{\frac{5}{3}}$, $\frac{n^2}{20} : \frac{q}{10} > 20$

Non-linear constraints. However, as we mentioned above, the Z3 solver does not directly support non-linear constraints in a way that is both fast and stable. Specific constraints such as $\log_2$, primality, and ensuring a variable is a power of two must be supported, because they appear very often in practice. We therefore added selected non-linear constraints with solver-friendly encodings. This design avoids a custom constraint engine, preserves usability on the CLI, and maintains reproducibility of runs. Now we introduce our selected non-linear constraints.

A primality test can be defined with the flag `-c` as `"is_prime(<var_name>)"`. Since our domains may go up to 2^{500} , using a global list of primes or brute-force trial division becomes infeasible. We have implemented a primality check with a two-stage approach. First, we add a pre-constraint to filter obvious composites using a list of relatively small integers (up to 5000), which shrinks the search space for Z3. Second, after the solver returns a candidate solution, we perform an exact primality test on this candidate. If the found value is composite (i.e., not a prime number), we add a blocking constraint forbidding that exact value for the variable, and perform a new Z3 search. This approach works well even for huge domains without using a massive prime table, and it keeps solving fast.

Another useful supported constraint is enforcing a variable to be a power of 2. This constraint can be defined with `-c "is_power_of_two(<var_name>)"`. We realize this

by constructing a finite list of powers of two in the relevant domain and giving it to Z3. Even for the largest domain we support, this set is small enough that we can afford it. The list is generated with bit shifting, which is constant-time per element and avoids any heavy arithmetic.

Furthermore, many cryptographic schemes or theoretical proofs to ensure the hardness of the problem use logarithmic relations. Our program supports the logarithmic function to base 2, which is the common base for logarithms in cryptographic area. Such a constraint can be defined with `-c "log2(<var_name>)"`. We model $\log_2(x)$ using a linear approximation, namely a piecewise-linear lower bound using secants. We partition the domain into breakpoints and attach a helper variable y with constraints of the form $y = m * x + b$ with precomputed slope m and intercept b on each domain segment. The segments are chosen so that the maximum deviation stays bounded to preserve precision of computations. For the largest domains we realize a dynamically set max error (up to ± 0.737) to keep the number of segments reasonable while maintaining a safe lower bound. To avoid recomputation, we precomputed tables (as CSV files) and cache them in memory once loaded. This allows us to reuse them efficiently across constraints. If the precomputed file is missing, the required data will be computed and cached during at runtime. With logarithms there is only one restriction from our side: the argument must be a single variable, not an expression. However, we provide a way to bypass this restriction, which we describe in the next paragraph below.

Custom variables and Big mode. In practice it is convenient to introduce auxiliary parameters that encapsulate relations between the core variables. This keeps constraint expressions readable and lets us reuse the same relation in multiple places. We support dynamical custom variables with the flag `-v` with typed definitions, namely `-v {int | real} <var_name> = <expression>`. For example, with `-v "int i = n * 10"` we define a new integer variable i . A new real variable r can be defined with `-v "real r = q ** (1 / 3)"`. The only limitation in naming is we cannot name our custom variable like `log<var_name>`, because naming beginning with `log` and following with a variable name is reserved by our logarithmic constraints parser. Once defined, these variables are fully integrated and can be used anywhere the predefined variables are used. In essence, custom variables make relation definitions modular and keep the CLI inputs maintainable. This mechanism also provides a clean bypass for the logarithmic restriction that an argument must be a single symbol. We can first pack the intended expression into a named variable with the `-v` flag and then apply the logarithmic constraint to that variable. The example of such an input can be seen in Listing 4.3.2.

```

1  python3 finder.py LWE -bit 40 -n n -q q -t t -v "real v = q
   / n" -c "log2(v) < 3"
2  # Output

```

Listing 4.3.2: Constraints input for the LWE problem.

Some schemes require very large domains. For example, SEAL configurations [LP16] may use $n = 8192$ for the LWE dimension with a modulus $q = 2^{383} - 2^{33} + 1$, and Dilithium5_MSIS [LDK⁺20] may use an MSIS total dimension $n = 256 \cdot 8$ with a modulus $q = 8380417$. To support such scenarios we provide the special flag `-enable-big` which toggles a big mode. In big mode we adjust domains and predefined intervals as presented in Table 4.2. The main challenge here was to preserve computational speed despite the larger ranges. We addressed this by using precomputed and cached secant segments for $\log_2$ constraints and our approach for primality check, and relying on our windowed search with adaptive step sizes.

Adjustment	Big mode	Normal mode
<i>Parameter ranges</i>		
Dimension n	$n \leq 35\,000$	$n \leq 1\,251$
Modulus q	$q \leq 2^{500}$	$q \leq 2^{18}$
Distribution parameter (Gaussian, real t)	$t \leq 8.0$	$t \leq 3.5$
Distribution parameter (other, integer t)	$t \leq 33$	$t \leq 10$
<i>Window base steps (initial Δ)</i>		
Base step Δ_n	100	30
Base step Δ_q	2 000	200
Base step Δ_t (Gaussian)	0.5	0.5
Base step Δ_t (other)	1	1

Table 4.2: Parameter ranges and initial window base steps in Big mode vs. Normal mode. Windows expand adaptively when no feasible/secure point is found.

Noise distributions. Some LWE-based cryptographic schemes require specific error and secret noise distributions. We let the user choose the distributions on the CLI. For the error term we support: centered binomial, discrete Gaussian, and Uniform mod. The error distribution can be selected with

`-error-distribution { cbinomial | dgaussian | unimod }`. The meaning of the parameter `-t` in terms of standard deviation for each supported distribution is shown in Table 4.3. We do not allow every noise distribution provided by the lattice estimator for the error. Some are not suitable for this purpose. For example, we get from these distributions very sparse vectors with most entries equal to zero, or larger error values do not occur at all, meaning that this could be exploited by an adversary to guess samples.

The secret distribution also can be set with

`-secret-distribution { cbinomial | dgaussian | unimod | binary | ternary | sbinary | sternary }`. In addition to supported error term distributions, we provide binary and sparse binary, ternary and sparse ternary options.

The error parameter stated with `-t` is part of the search. The secret parameter we can optionally provide with the `-s {<int> | <real>}` flag. If either the secret or error distribution is omitted, centered binomial is used. If secret parameter is omitted, a predefined default is used. The defaults and minimal values for each secret distribution are given in Table 4.4. Minimal values are needed to avoid trivial instances. This design keeps the interface simple.

Distribution	Standard deviation σ
Centered Binomial	$\sqrt{\frac{t}{2}}$
Discrete Gaussian	t
Uniform Mod	$\sqrt{\frac{t^2-1}{12}}$

Table 4.3: Standard deviation for supported error term distributions expressed via the parameter t .

Secret distribution	Default $-s$ value	Minimum $-s$ value
Binary	<i>ignored</i>	<i>ignored</i>
Ternary	<i>ignored</i>	<i>ignored</i>
Sparse Binary	16	4
Sparse Ternary	16	4
Centered Binomial	2	1
Uniform Mod	5	2
Discrete Gaussian	1.5	0.5

Table 4.4: Default and minimum secret parameters (flag `-s`). Distributions marked “ignored” do not accept `-s`; if provided, it is ignored.

Length of the SIS input. With the optional flag `-m <integer>` we can set the required length of the SIS input. However, lower input length can lead to an increase in other parameters.

Optimization goals. In many schemes we do not only need a secure problem instance with values which meet the constraints, but also want to minimize a parameter for efficiency. We allow objective functions so the solver not only finds a satisfying model but also minimizes a chosen variable or even an expression. Objectives can be added with `-opt {<var_name> | <expression>}`. The example of usage and how the final output differs can be seen in Listing 4.3.3.

As we have mentioned, objectives can contain several variables and expressions. We can pass several just by using multiple `-opt` flags and expressions must be packed in a string. For example, `-opt "q / 10" -opt "log2(n) + q / n" -opt n`.

The priority for multiple objectives can be selected with `-opt-priority {lex | pareto | box}`. Using lexicographic method we set a priority order: optimize the first objective as much as possible, then the second, and so on. With box optimization we shrink all objectives fairly at the same time. Z3 looks for a model that meets all tightened bounds together. Pareto-optimal optimization prefer solutions that improve any objective without worsening others.

```

1  python3 finder.py MLWE -bit 40 -N N -n n -d d -q q -t t -c "n
2    % 2 == 0" -c "d > 1" -c "d < 10"
3  # Output
4  All attacks passed threshold!
5  N = 108
6  d = 9
7  n = 12
8  q = 700
9  t = 2
10 -----
11 python3 finder.py MLWE -bit 40 -N N -n n -d d -q q -t t -c "n
12    % 2 == 0" -c "d > 1" -c "d < 10" -opt d
13 # Output
14 All attacks passed threshold!
15 N = 108
16 d = 2
17 n = 54
   q = 700
   t = 2

```

Listing 4.3.3: Optimization usage on the MLWE problem.
Input and the final solution.

Refinement. With smaller search windows we could reach more precise parameter values. However, there is a trade-off: smaller windows can give more precise results, but they require many more estimator calls, which leads to increased runtime of the program. Nonetheless, in some cases, we want to get more precise values in exchange for runtime.

By default, to provide faster computation, our windowed search uses larger windows, which can produce some overkill (parameters larger than necessary). Once we have reached the target security, we sometimes want to reduce this overkill. With the `-enable-refine` flag we run a post-processing refinement step once a secure parameter set has been found. In essence, if a parameter contributes more bit security for some attacks than the specified margin, we try to shrink that parameter while keeping the required security level. The security margin is set with `-bit-sec-margin <integer>`.

By default, if refinement is enabled, the security margin is 10. The lower the margin, the more refinement attempts and the more computational time is required.

During the implementation we faced specific challenges. We need to reduce overkill without going below the target, respect all constraints, handle integer and real value mixes, and avoid turning refinement into a second expensive global search. After the secure point is found, we run a bounded binary search per overkill parameter. Namely, we set a local interval for each candidate parameter from the previous window bound up to the current value. We use Z3 to enforce all constraints and fix the other parameters at the secure values. For the parameter we refine at the current step, we shrink bounds step by step in a half-interval strategy. At each midpoint we ask Z3 for a feasible solution, and if one is found we call the estimator to check the security. If still secure enough, we move the upper bound down, otherwise we move the lower bound up. We keep the best secure point found, set it as the value of the corresponding parameter for the remaining refinement steps, and proceed to the next parameter.

Refinement adds some estimator calls, but only inside a small local interval, not the full domain. Thus, we keep the speed benefits of windowed search and recover precision where it matters, under a controlled security margin. Also, all constraints remain fulfilled, and results stay deterministic and auditable. The example usage and the final result of the refinement step can be seen in Listing 4.3.4. There we obtained more precise values for the LWE dimension and the modulus, while meeting the required bit security.

```

1  python3 finder.py LWE -bit 50 -n n -q q -t t
2  # Output
3  Bit security: {'usvp': 60.5, 'dual': 62.7, 'dual_hybrid':
4  60.5, 'bkw': 68.3, 'arora-gb': 59.8, 'bdd': 57.7}
5  All attacks passed threshold!
6  n = 161
7  q = 700
8  t = 2
9  -----
10 python3 finder.py LWE -bit 50 -n n -q q -t t --enable-refine
11 --bit-sec-margin 8
12 # Output
13 Z3 solution found: n=161, q=700, t=2
14 Bit security: {'usvp': 60.5, 'dual': 62.7, 'dual_hybrid':
15 60.5, 'bkw': 68.3, 'arora-gb': 59.8, 'bdd': 57.7}
16 All attacks passed threshold!
17 n = 161
18 q = 700
19 t = 2
20 Binary search testing...
21 # Final output
22 Tested (141, 500, 2) -> {'usvp': 57.1, 'dual': 59.3, '
23 dual_hybrid': 58.1, 'bkw': 58.0, 'arora-gb': 57.9, 'bdd':
24 54.3}
25 Refined params: n=141, q=500, t=2

```

Listing 4.3.4: Refinement usage on the LWE problem.

5 Evaluation

5.1 Criteria and Improvements

We evaluated our program with simple criteria: computational effort, runtime, precision, robustness under small input changes, and scalability to wider domains and tighter constraints.

Computational effort and runtime. In early versions we focused on finding a practical trade-off between precision and effort to keep runtime low. We tested different search strategies and measured total runtime. Thanks to several optimizations: windowed search, sensible step sizes (which allow to reach the right search space faster), precomputations and cached results, and optimized Z3 models, we reached a good balance. In most profiles the estimator dominates the runtime. Also we restrict domains with a parameter-specific upper-bound, which allows us to perform computations faster. If parameters have to be big, we provide an optional big-number mode, which increases the domains.

Precision. Returned parameters consistently meet the target across the standard attack set. In typical runs the bit security margin above the target is small, which was the goal. Namely, we aimed to avoid oversizing while still being safe. For users who need tighter results, an optional refinement step is provided to achieve more precise values at the cost of extra estimator calls.

Robustness. Our program is fully deterministic. The same input always lead to the same result. Small changes to inputs lead to similar solutions with similar runtime. Possible system exceptions are handled properly to avoid unpredictable behaviour. The verbose flow of the program is printed in the command line to allow the user to observe the process.

Scalability. As domains grow, the program scales by expanding windows rather than enumerating everything. Furthermore, the selected non-linear constraints are designed to preserve computation speed. Precomputations and cached data also help a lot.

Improvements. If we had more time for the survey and the development, we would add explicit exponent constraints (e.g., $q = 2^l$, where l is some custom variable), because some schemes naturally use them. We would also use accumulated data to build data-driven precomputations, closer start lower bounds in certain domains, to cut estimator call

further. More estimator fine-tuning would let users align runs with specific cost models. Finally, deeper caching and parallelism would reduce runtime on very large instances.

5.2 Practical Implications.

The program makes parameter choice much faster and more reliable than manual probing. It checks all relevant attacks, respects user constraints, avoids overkill, and produce transparent and clear outputs that are easy to audit. In practice, this means a reproducible path from the input to concrete values. All this leads to faster deployment of cryptographic instances for concrete schemes.

6 Installation requirements

This chapter lists the software prerequisites required by the program.

- **Python** (recommended: 3.10) and **git**.
- **SymPy** (Python package).
- **Z3** Python bindings (**z3-solver**).
- **SageMath** (provides the **sage** command and the **sageall** Python module).
- **lattice-estimator** (GitHub checkout): <https://github.com/malb/lattice-estimator/tree/main>.
- Our provided source files and folders (to be copied into the estimator's root).

Bibliography

- [ABD⁺19] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber algorithm specifications and supporting documentation. Manuscript, 2019. NIST PQC Round 2 documentation.
- [ABF⁺20] Martin R. Albrecht, Shi Bai, Pierre-Alain Fouque, Paul Kirchner, Damien Stehlé, and Weiqiang Wen. Faster enumeration-based lattice reduction: Root Hermite factor time. In *Annual International Cryptology Conference*, pages 186–212. Springer, 2020.
- [ACD⁺18] Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the LWE and NTRU schemes! In *International Conference on Security and Cryptography for Networks*, pages 351–367. Springer, 2018.
- [ACW19] Martin R Albrecht, Benjamin R Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In *International Conference on Selected Areas in Cryptography*, pages 467–491. Springer, 2019.
- [AD17] Martin R. Albrecht and Amit Deo. Large modulus Ring-LWE $\geq$ Module-LWE. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 267–296. Springer, 2017.
- [AFG13] Martin R. Albrecht, Robert Fitzpatrick, and Florian Göpfert. On the efficacy of solving LWE by reduction to unique-SVP. In *International Conference on Information Security and Cryptology*, pages 293–310. Springer, 2013.
- [AG11] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In *Automata, Languages and Programming*, pages 403–415, Berlin, Heidelberg, 2011. Springer.
- [Ajt96] Miklós Ajtai. Generating hard instances of lattice problems. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 99–108. ACM, 1996.

- [APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.
- [BBB⁺22] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, pages 415–442, Cham, 2022. Springer International Publishing.
- [BDK⁺18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [BGV14] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory*, 6(3):1–36, 2014.
- [BKW03] Avrim Blum, Adam Kalai, and Hal Wasserman. Noise-tolerant learning, the parity problem, and the statistical query model. *Journal of the ACM*, 50(4):506–519, 2003.
- [BLL⁺22] Lei Bi, Xianhui Lu, Junjie Luo, Kunpeng Wang, and Zhenfei Zhang. Hybrid dual attack on LWE with arbitrary secrets. *Cybersecurity*, 5(1):15, 2022.
- [BLLW22] Lei Bi, Xianhui Lu, Junjie Luo, and Kunpeng Wang. Hybrid dual and Meet-LWE attack. In *Information Security and Privacy*, pages 168–188, Cham, 2022. Springer.
- [Bra66] Auguste Bravais. *Études cristallographiques : Mémoire sur les systèmes formés par des points distribués régulièrement sur un plan ou dans l’espace*. Gauthier-Villars, Paris, 1866.
- [CHHS19] Jung Hee Cheon, Minki Hhan, Seungwan Hong, and Yongha Son. A hybrid of dual and meet-in-the-middle attack on sparse and ternary secret LWE. *IEEE Access*, 7:89497–89506, 2019.
- [CN11] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 1–20. Springer, 2011.
- [CS13] John Horton Conway and Neil James Alexander Sloane. *Sphere packings, lattices and groups*, volume 290. Springer, 2013.

- [Ded79] Richard Dedekind. *Über die Theorie der ganzen algebraischen Zahlen*. Vieweg, Braunschweig, 1879. Supplement XI to Dirichlet's *Vorlesungen über Zahlentheorie*.
- [DMB08] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS'08/ETAPS'08, pages 337–340, Berlin, Heidelberg, 2008. Springer-Verlag.
- [EJK20] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a dual/hybrid approach to small secret LWE: A dual/enumeration technique for learning with errors and application to security estimates of FHE schemes. In *International Conference on Cryptology in India*, pages 440–462. Springer, 2020.
- [Für07] Martin Fürer. Faster integer multiplication. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing*, pages 57–66. ACM, 2007.
- [Gau01] Carl Friedrich Gauss. *Disquisitiones arithmeticae auctore D. Carolo Fridrico Gauss*. in commissis apud Gerh. Fleischer, jun., 1801.
- [GJ21] Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving LWE with applications to CRYSTALS. In *Advances in Cryptology – ASIACRYPT 2021*, pages 33–62, Cham, 2021. Springer.
- [GJS15] Qian Guo, Thomas Johansson, and Paul Stankovski. Coded-BKW: Solving LWE using lattice codes. In *Annual Cryptology Conference*, pages 23–42. Springer, 2015.
- [KF15] Paul Kirchner and Pierre-Alain Fouque. An improved BKW algorithm for LWE with applications to cryptography and lattices. In *Annual Cryptology Conference*, pages 43–62. Springer, 2015.
- [KNK20] Zahyun Koo, Jong-Seon No, and Young-Sik Kim. Reduction from module-SIS to ring-SIS under norm constraint of ring-SIS. *IEEE Access*, 8:140998–141006, 2020.
- [Latnd] Lattice Estimator Development Team. Lattice estimator documentation, n.d.
- [LDK⁺20] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-Dilithium: Algorithm specifications and supporting documentation, 2020. NIST Post-Quantum Cryptography Standardization Round 3 documentation.

- [LN13] Mingjie Liu and Phong Q. Nguyen. Solving BDD by enumeration: An update. In *Cryptographers' Track at the RSA Conference*, pages 293–309. Springer, 2013.
- [LNP22] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: shorter, simpler, and more general. In *Annual International Cryptology Conference*, pages 71–101. Springer, 2022.
- [LP16] Kim Laine and Rachel Player. Simple encrypted arithmetic library—SEAL (v2.0). Technical report, 2016.
- [LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology—EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 1–23. Springer, 2010.
- [LS14] Antoine Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2014.
- [Min96] Hermann Minkowski. *Geometrie der Zahlen*. Teubner, Leipzig, 1896.
- [MR09] Daniele Micciancio and Oded Regev. Lattice-based cryptography. In Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen, editors, *Post-Quantum Cryptography*, pages 147–191. Springer, Berlin, Heidelberg, 2009.
- [NHAJRM24] Huda Naeem Hleeb Al-Jabbari, Ali Rajaei, and Abbas Maarefparvar. A digital signature scheme based on Module-LWE and Module-SIS. *arXiv e-prints*, 2024. arXiv preprint.
- [NV10] Phong Q. Nguyen and Brigitte Vallée. *The LLL algorithm*. Springer, 2010.
- [Pei15] Chris Peikert. What does GCHQ’s “cautionary tale” mean for lattice cryptography. Manuscript, 2015.
- [Pei16] Chris Peikert. A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4):283–424, 2016.
- [PR06] Chris Peikert and Alon Rosen. Efficient collision-resistant hashing from worst-case assumptions on cyclic lattices. In *Theory of Cryptography Conference*, pages 145–166. Springer, 2006.
- [QLZ⁺23] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Mingyao Shao, and Shuo Sun. When NTT meets SIS: Efficient side-channel attacks on Dilithium and Kyber, 2023. Paper 2023/XXXX.

- [Reg04] Oded Regev. Lattices in computer science, 2004. Lecture notes, Tel Aviv University.
- [Reg09] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
- [Wal56] I. Waller. Dynamical theory of crystal lattices by m. born and k. huang. *Acta Crystallographica*, 9(10):837–838, 1956.
- [Wun18] Thomas Wunderer. *On the security of lattice-based cryptography against lattice reduction and hybrid attacks*. Doctoral dissertation, Technische Universität Darmstadt, 2018.
- [Wun19] Thomas Wunderer. A detailed analysis of the hybrid lattice-reduction and meet-in-the-middle attack. *Journal of Mathematical Cryptology*, 13(1):1–26, 2019.
- [WX22] Han Wu and Guangwu Xu. Enhancing the dual attack against MLWE: Constructing more short vectors using its algebraic structure. *Cryptology ePrint Archive*, 2022. Paper 2022/XXXX.
- [ZZW22] Xue Zhang, Zhongxiang Zheng, and Xiaoyun Wang. A detailed analysis of primal attack and its variants. *Science China Information Sciences*, 65(3):132301, 2022.